

HACETTEPE UNIVERSITY

Department of Nuclear Engineering

NEM 394 ENGINEERING PROJECT III

Two-Dimensional Transient Thermal Analysis of a Rectangular Plate for Both Heating and Cooling Phases

Student No. : **2230386062**

Student Name : **Emre Sakarya**

Delivery Date: 29.05.2026

Due Date: 29.05.2026

1 ABSTRACT

In this project, the two-dimensional, time-dependent (transient) heat transfer of a rectangular plate heated by spring clamps at its four corners was investigated in two separate stages. The plate dimensions are 6 cm × 16 cm × 1 cm, and the study was performed on a 3x5 node grid with $\Delta x = 3$ cm and $\Delta y = 4$ cm intervals. The open finite difference method based on the control volume approach was used as the solution method. Furthermore, both solid and liquid metal coolant properties were updated at each time step depending on the node temperature; they were not considered constant.

In the first stage, starting at 300 K, the plate was subjected to four local and variable volumetric heat generation points (28–37 kW/m³) via clamps at its four corners and heated under isolated boundary conditions until all nodes reached at least 1000 K. The maximum time step for this calculation was estimated to be approximately **102** seconds. It took approximately **28.643 seconds, or almost eight hours**, for all nodes to exceed the 1000 K threshold.

In the second stage, the plate was immersed in a liquid metal bath, where the plate temperature was defined as a function of time, and the temperature decreased linearly over time. Unlike the first stage, no heat generation occurred in this stage; only natural convection boundary conditions were applied to the entire surface of the plate. The heat transfer coefficient was calculated for each node and each time step from correlations based on the Rayleigh number. Due to the high Biot number, the stability limit of the cooling stage dropped to approximately **0.043** seconds. All nodes reached below 600 K in approximately **1920.3 seconds or about 32 minutes**. Finally, the microscopic temperature delay (0.0384 K) observed between the solid plate and the liquid bath at the end of the cooling process confirms that the plate's internal conduction resistance substantially exceeds the surface convection resistance ($Bi \gg 1$), placing the system well outside the lumped-capacitance regime.

2 INTRODUCTION

Time-dependent (transient) two-dimensional heat transfer problems are of great importance in many critical engineering fields, from the thermal design of nuclear reactor components to power systems, cooling systems, and aerospace applications. In real-world applications, the thermophysical properties of materials (thermal conductivity, specific heat, and density) do not remain constant but continuously change with temperature, leading to highly nonlinear differential equations governing the system. This variability makes it impossible to obtain exact analytical solutions. Therefore, the use of numerical solution methods such as the Finite Difference Equation (FDM) or the Control Volume Equation (CVFDM) is extremely important and vital.

This project investigates a two-stage heat treatment process for a rectangular plate. In the first stage, the plate is heated by volumetric heat generation at varying intensities via spring clamps placed at its four corners. All nodal points must reach at least 1000 K for the plate to become usable. In the second stage, the treated plate is transferred to a liquid metal bath, this time without a heat source, and cooled to below 600 K by natural convection.

To solve the problem numerically, an **explicit** finite difference method based on control volume formulation was chosen. The main reason for choosing this method is that the node temperatures can be updated independently at each time step, thus allowing temperature-dependent material properties to be easily incorporated into the system at each step.

In this study, the plate geometry is divided into a node network consisting of a total of 15 nodes with dimensions of 3 × 5. In this study, as in real life, the thermophysical properties of both the solid plate and the liquid metal coolant were **not assumed to be constant**, but were updated according to the instantaneous temperature values for each node. During the cooling phase, the natural convection coefficient was determined using the Rayleigh number, calculated from the local temperature difference, with separate correlations for horizontal surfaces and vertical edges. The combination of all these approaches allowed us to obtain a realistic and consistent numerical model.

3 METHODOLOGY

Nodal Network Structure and Control Volume Definitions:

The plate geometry has dimensıons L_x = 0.06 m and L_y = 0.16 m, and is discretized with spacing dimensions Delta x = 0.03 m and Delta y = 0.04 m. This creates a two-dimensional mesh structure consisting of 15 nodes, with n_x = 3 in the x-direction and n_y = 5 in the y-direction. According to the Finite Difference Control Volume Method the volumetric dimensions of the boundary and corner nodes are different from the volumetric dimensions of the interior nodes. The widths (i,j) of each node are defined by the following functions:

$$\Delta x_n(i) = \begin{cases} \Delta x/2, & i = 0 \text{ or } i = n_x - 1 \\ \Delta x, & \text{other cases} \end{cases}$$

$$\Delta y_n(j) = \begin{cases} \Delta y/2, & j = 0 \text{ or } j = n_y - 1 \\ \Delta y, & \text{other cases} \end{cases}$$

Based on these calculated widths, the control volume V(i,j) for each node of the plate with a thickness of L_z = 0.01 m is calculated using the following function:

$$V_{i,j} = \Delta x_n(i) \cdot \Delta y_n(j) \cdot L_z$$

Modelling of Volumetric Heat Source

$$\dot{q}_{0,\ 0} = 32 \times 10^3\ \mathrm{W/m^3}$$
$$\dot{q}_{n_x-1,\ 0} = 35 \times 10^3\ \mathrm{W/m^3}$$
$$\dot{q}_{0,\ n_y-1} = 28 \times 10^3\ \mathrm{W/m^3}$$
$$\dot{q}_{n_x-1,\ n_y-1} = 37 \times 10^3\ \mathrm{W/m^3}$$

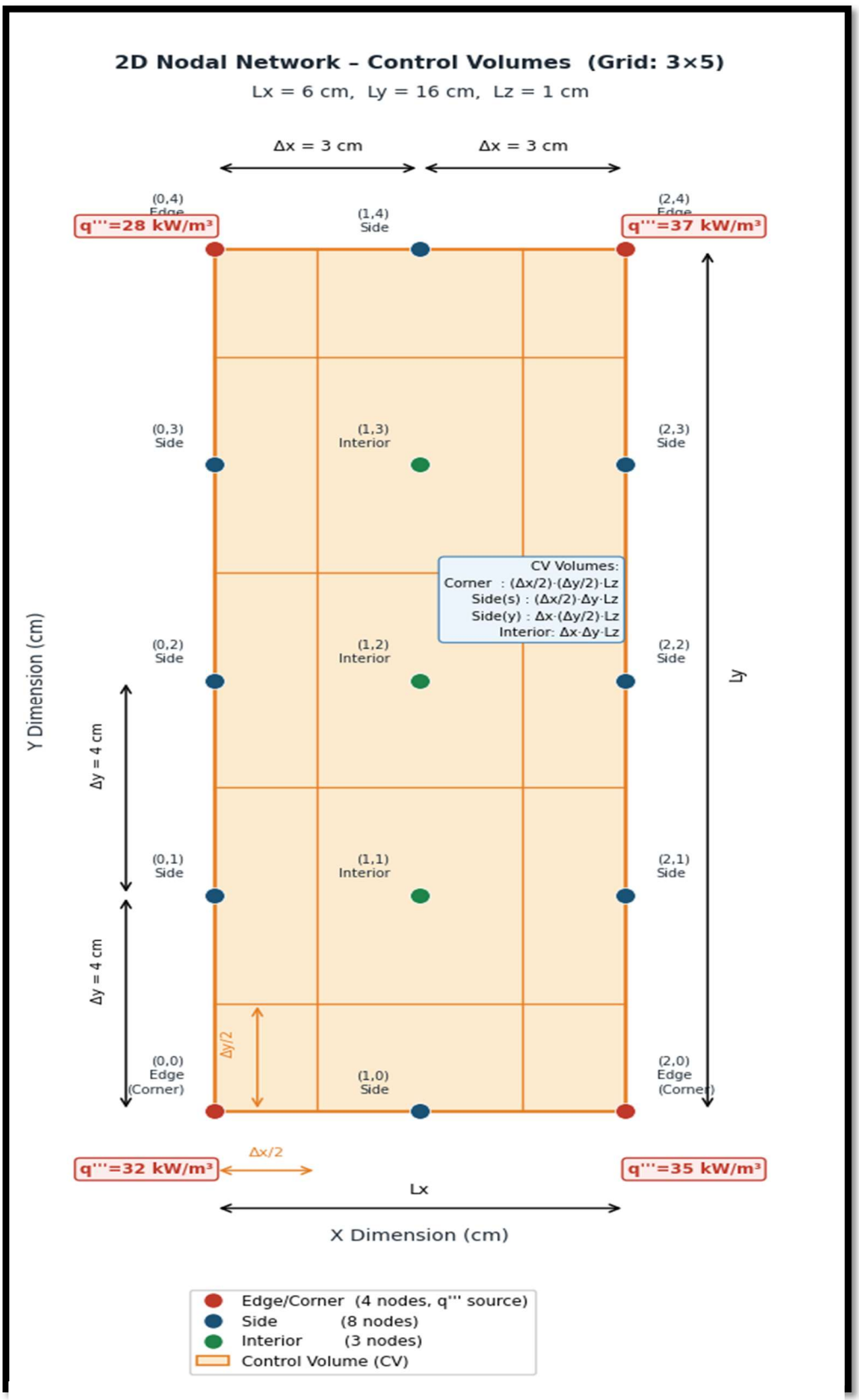


Figure 1: Two-dimensional 3x5 nodal grid network and control volume (CV) spatial definitions for the rectangular plate geometry

There is no heat source at any of the interior and edge points outside these nodes; therefore, the volumetric heat generation term is equal to zero.

Numerical Formulation of the Heating Section

During the heating process, all outer boundaries of the plate are closed, i.e., an adiabatic boundary condition is assumed. The explicit Euler method was applied in time integration. Temperature-dependent variable solid material properties are updated at each time step.

$$T_{i,j}^{n+1} = T_{i,j}^{n} + \frac{\Delta t}{\rho_s\left(T_{i,j}^{n}\right)\cdot c_{p,s}\left(T_{i,j}^{n}\right)\cdot V_{i,j}}\left[\dot{Q}_{x-} + \dot{Q}_{x+} + \dot{Q}_{y-} + \dot{Q}_{y+} + \dot{q}_{i,j}V_{i,j}\right]$$

Heat fluxes ($\dot{Q}$) generated by conduction between adjacent nodes were modeled by taking the arithmetic mean of the thermal conductivities at the interfaces and using Fourier's Law of Conduction.

$$\dot{Q}_{x+} = \left(\frac{k_s\left(T_{i,j}^{n}\right) + k_s\left(T_{i+1,j}^{n}\right)}{2}\right)\cdot\left(\Delta y_n(j)\cdot L_z\right)\cdot\frac{T_{i+1,j}^{n} - T_{i,j}^{n}}{\Delta x}$$

Heating section Stability Criterion:

To prevent deviation in the analytical solution, the time step is limited to the highest operating temperature at which the thermal emissivity coefficient (α_s) reaches its maximum, i.e., 1000K.

$$\Delta t_{max,heating} = \frac{1}{2\cdot\alpha_{s,max}\cdot\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right)}$$

$$\alpha_{s,max} = \frac{k_s(1000\ \mathrm{K})}{\rho_s(1000\ \mathrm{K})\cdot c_{p,s}(1000\ \mathrm{K})}$$

Formulation of the Cooling Part in Liquid (T_bath)

$$T_{bath}(t) = 1000.0 - 125.0\cdot\left(\frac{t}{600.0}\right)$$

All six surfaces of the plate have also been subjected to natural convection. The nodal energy equation for this situation is as follows:

$$T_{i,j}^{n+1} = T_{i,j}^{n} + \frac{\Delta t}{\rho_s(T_{i,j}^{n}) \cdot c_{p,s}(T_{i,j}^{n}) \cdot V_{i,j}} \left[\sum \dot{Q}_{conduction} - \dot{Q}_{convection} \right]$$

Local total surface convective heat flux

$$\dot{Q}_{convection} = (h_{top} + h_{bottom}) \cdot A_{face} \cdot (T_{i,j}^{n} - T_{bath}^{n}) + \sum \left(h_{vert} \cdot A_{side} \cdot (T_{i,j}^{n} - T_{bath}^{n}) \right)$$

Natural Convection Correlations and Characteristic Lengths

Rayleigh number (Ra):

$$Ra = \frac{g \cdot \beta_l \cdot |T_{i,j} - T_{bath}| \cdot L_c^3}{\nu_l \cdot \alpha_l}$$

Characteristic length for horizontal surfaces:

$$L_{c,h} = \frac{A_s}{P} = \frac{L_x \cdot L_y}{2 \cdot (L_x + L_y)} = 0.02182 \text{ m}$$

Nusselt correlation for horizontal top surface:

$$Nu = \begin{cases} 0.54 \cdot Ra^{0.25}, & Ra < 10^7 \\ 0.15 \cdot Ra^{1/3}, & Ra \geq 10^7 \end{cases}$$

Nusselt correlation for horizontal bottom face:

$$Nu = 0.27 \cdot Ra^{0.25}$$

Churchill and Chu correlation for vertical section edges (Nu):

$$Nu = \left(0.825 + \frac{0.387 \cdot Ra^{1/6}}{\left(1 + \left(\frac{0.492}{Pr_l}\right)^{9/16}\right)^{8/27}} \right)^2$$

Dynamic Stability Analysis:

Combined convection + conduction dynamic time limit equation

$$\Delta t_{max,cooling} = \min_{i,j} \left(\frac{\rho_s \cdot c_{p,s} \cdot V_{i,j}}{\sum g_{conduction} + HA} \right)$$

Here, the expression $\sum g_{conduction}$ in the denominator represents the sum of the conduction coefficients of neighboring nodes, while the HA term represents the sum of all convection distances the node is exposed to:

$$\boldsymbol{HA = (h_{top} + h_{bottom}) \cdot A_{face} + \sum (h_{vert} \cdot A_{side})}$$

The actual safe cooling time step applied.

$$\boldsymbol{\Delta t_{cooling_actual} = 0.40 \cdot \Delta t_{max,cooling}}$$

4 RESULTS

4.1 Results of The Heating Part

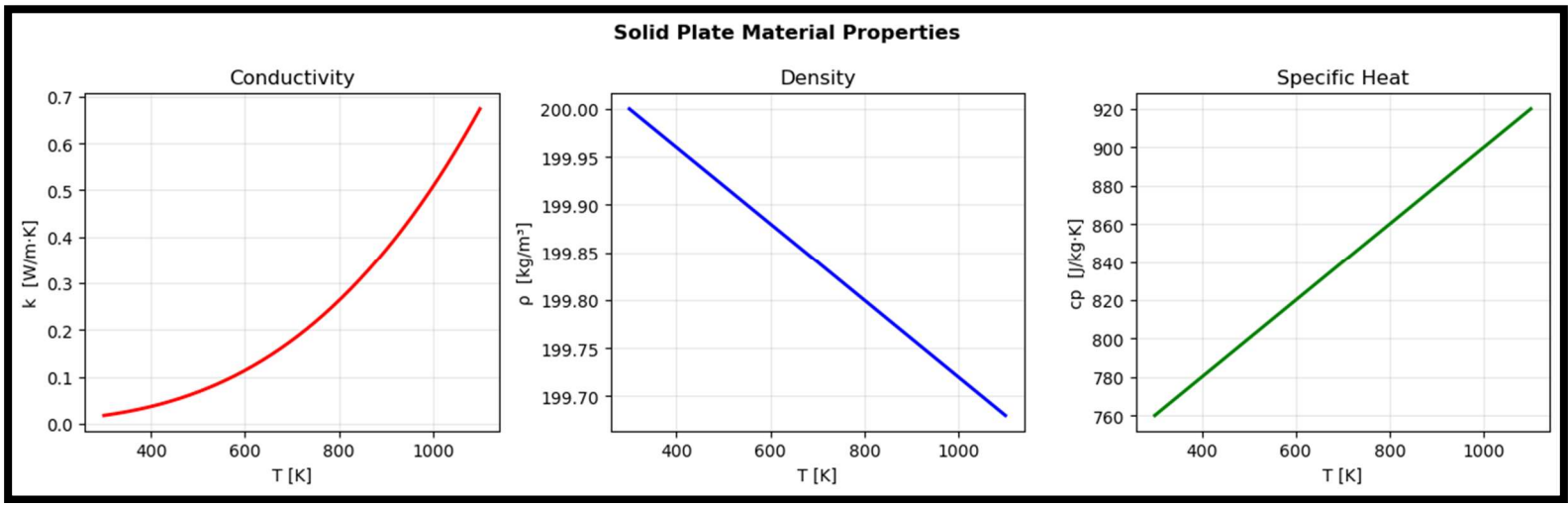


Figure 2: Solid plate material properties as a function of temperature

4.1.1 Thermophysical Properties of Solid Plates

Throughout the simulation, the thermal conductivity, density, and specific heat of the solid plate were updated at each time step according to the corresponding node temperature, which is the sole factor ensuring the solution's realism. Figure 1 above shows how these three properties change with temperature. The thermal conductivity k(T) increases from **0.0175 W/m·K at 300 K to 0.5075 W/m·K at 1000 K** due to the cubic term being temperature-dependent; this approximately 29 times increase is directly reflected in the dynamics of the simulation. Density, on the other hand, remains almost constant due to the small coefficient of thermal expansion, decreasing only from **200 to 199.720 kg/m³** despite the 700 Kelvin increase. The specific heat, however, increased linearly from **760 J/kg·K to 900 J/kg·K.** These changes lead to an approximately 25 times increase in thermal diffusivity α(T) and determine the most critical condition of the stability criterion, as will be shown below.

4.1.2 Assessment of the Stability Criterion

Assessment of the Stability Criterion: When the stability condition described in the methodology section was evaluated under T = 1000 K conditions, the maximum allowable time step was obtained as approximately 102 seconds. A 90% confidence factor, i.e., Δt = 91.8 seconds, was used to guarantee stability in the simulation. Since the α value is much smaller at lower temperatures, it can be said that this time step initially remained quite conservative.

4.1.3 Temperature Variation of Corner Nodes

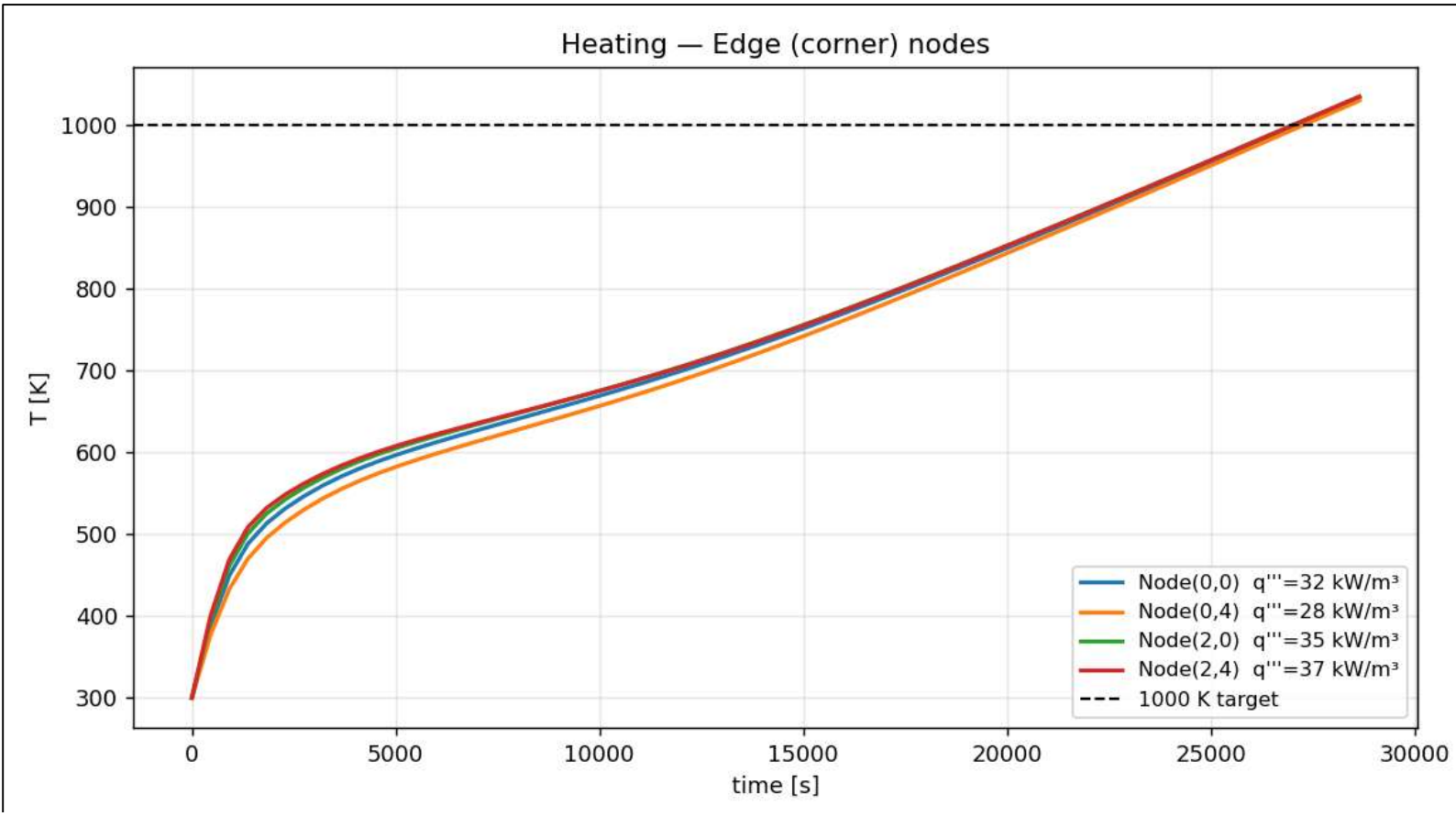


Figure 3: Temperature change at the edge (corner) nodes during heating.

Figure 3 shows the temperature variation of the four corner nodes. Each curve shows the coordinate of the respective corner and the applied q‴ value. As expected, Node (2,4) (37 kW/m³), where the highest volumetric heat generation is applied, was the fastest heating node; followed by corners with heat sources of 35, 32 and 28 kW/m³ respectively. The important part of the graph is as follows: It is observed that the curves are initially steep and have a gentle slope over time. The main reason for this behaviour is that the k(T) value increases rapidly with temperature, and therefore heat escape from the corners to neighbouring nodes via conduction strengthens over time. In other words, the heat initially accumulated at the corners is transferred to the environment more efficiently as the temperature increases, and the heat transfer in the system is balanced.

4.1.4 Temperature Variation of Edge (Side) Nodes

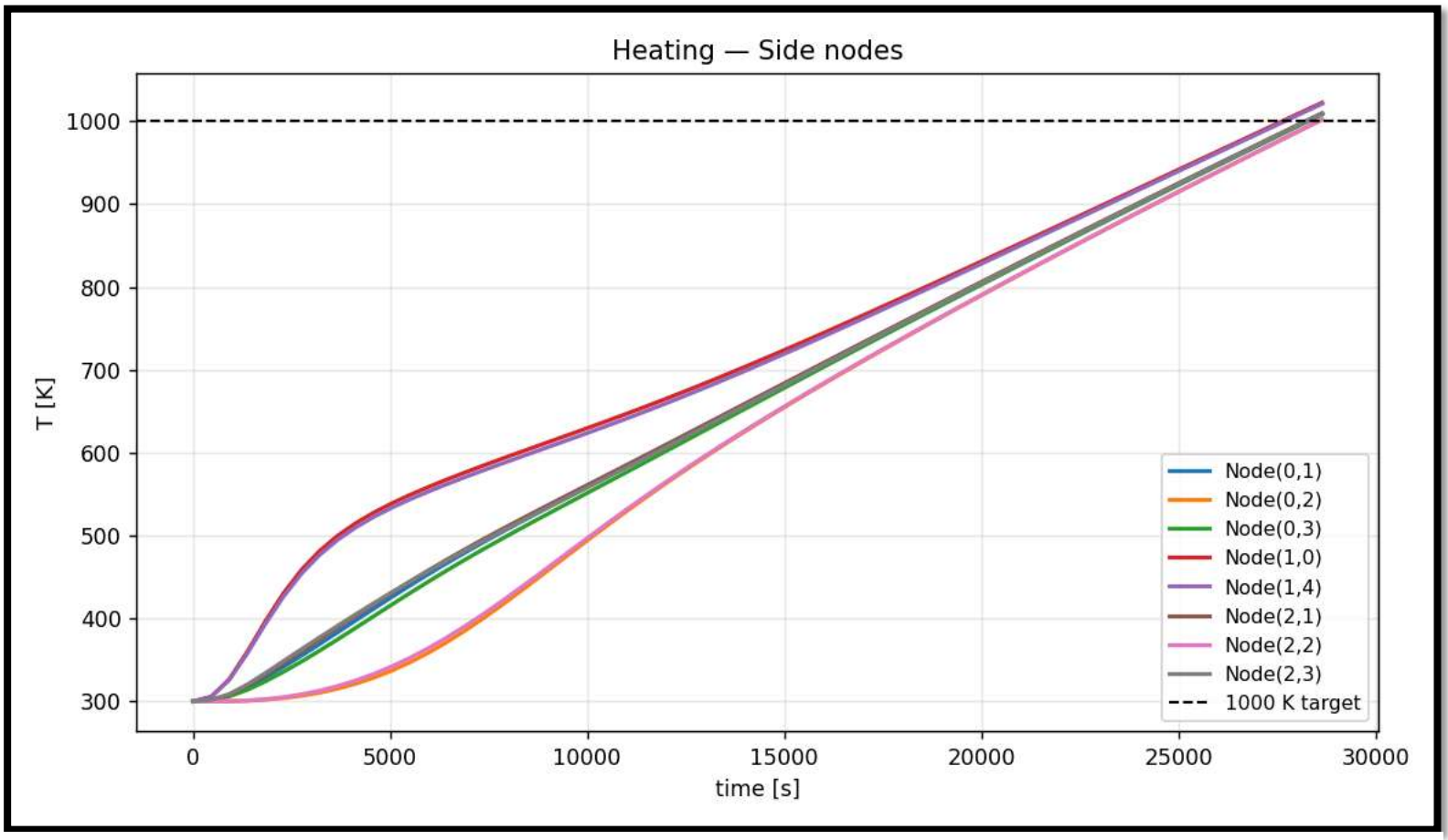


Figure 4: Temperature evolution at side nodes during heating.

The figure above shows the temperature history of eight edge nodes located on the outer edge of the plate but not directly exposed to heat generation. These nodes only receive heat through conduction from neighboring nodes over time. Significant differences were observed between the edge nodes depending on their location. Edge nodes adjacent to the heat source heated up faster. In contrast, nodes further away and not directly close to the source took longer to heat up. This difference is very clear in the first 5000 seconds. As a result, all edge nodes exceeded the 1000 K threshold in approximately 28000 seconds.

4.1.5 Temperature Variation of Internal Nodes

Figure 5 below shows the three inner nodes (Node (1,1), Node (1,2), Node (1,3)) located at the center of the plate. These nodes are equidistant from all heat sources and can only indirectly receive heat through their neighboring edge nodes. Therefore, heating is initially quite slow, and since the temperature of their non-heat source neighbours must also rise beforehand, the curves remain almost horizontal for the first 2000-3000 seconds. The

node that reaches 1000 K latest during the simulation is Node (1,2), which is exactly in the center because it is as far away from all four heat sources as possible. This result is exactly what was expected and desired, as it is consistent with the laws of physics. The simulation was terminated at **t = 28643 second ≈ 7.95 hours when this slowest heating node exceeded the 1000 K threshold, and this value is one of the most important results of the project.**

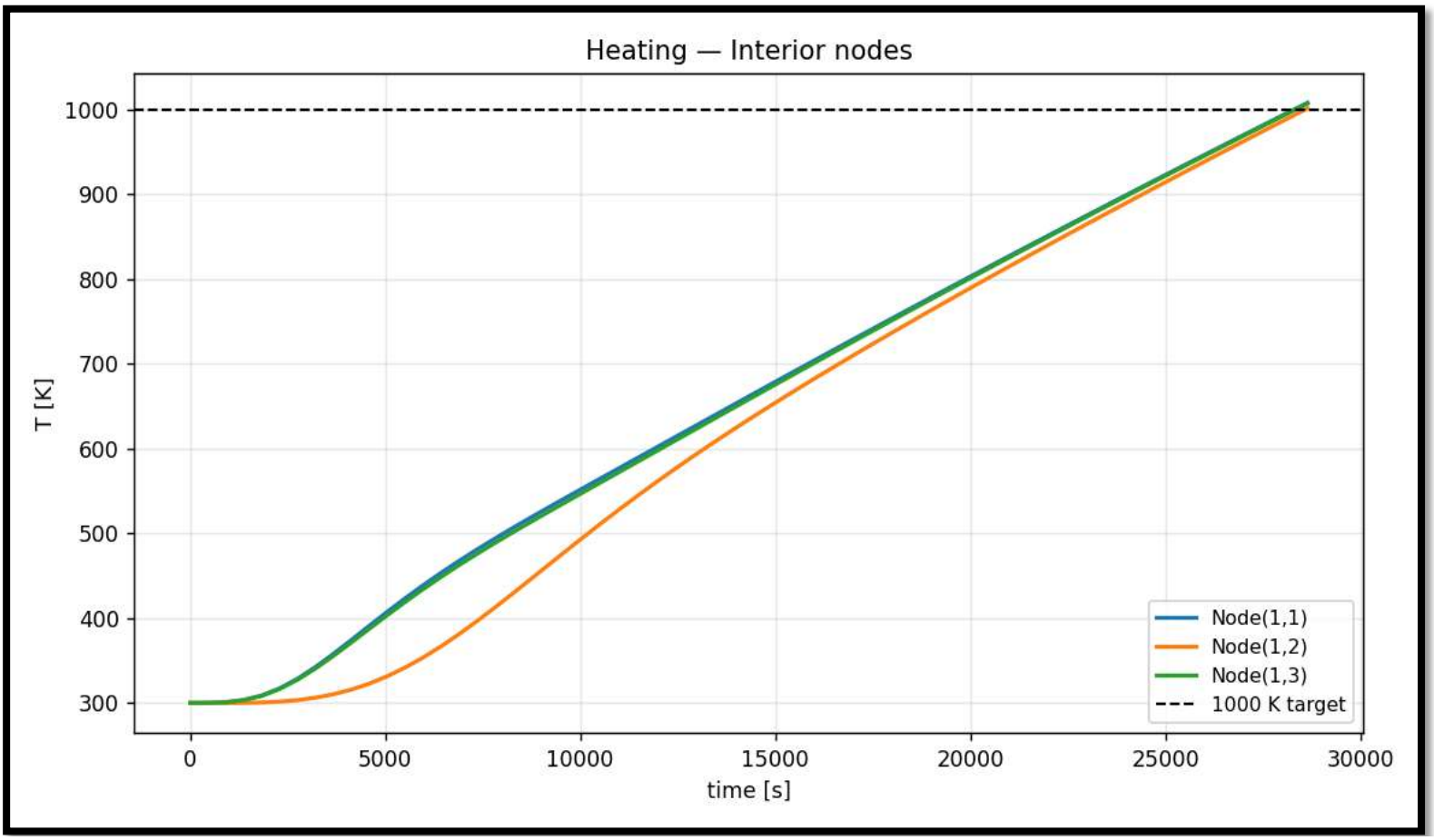


Figure 5: Temperature evolution at interior nodes during heating.

4.1.6 Comparing Different Time Step Values

In accordance with the project requirements, the simulation was repeated with different time steps, and the results are presented in two separate graphs in Figure 6. The left panel shows the graph of the total physically required time, while the right panel compares the number of iterations required on the logarithmic axis.

The most important observation from the results is that the physical time required to reach 1000 K is almost independent of the Δt value; it is approximately 28,580 ± 50 second in all trials. **The small deviation (28,643s) seen in the study with Δt = 91.8 s** is due to the decomposition sensitivity resulting from the final step slightly exceeding the 1000 K

threshold. Although the difference appears large in the graph, it is necessary to focus on the numerical values to avoid being misled by the diagram. I mention this because, although it appears small in the graph on the right, the difference is actually very large since the scale is logarithmic.

In contrast, the computational cost is inversely proportional to Δt. While a Δt = 101 s, close to the maximum limit, required only 283 time steps, choosing Δt = 0.1 s required 2.85777 x 10^5 steps; that is, approximately 1000 times more iterations to reach the same result. Very high Δt values near the stability limit may be weak against fluctuations exceeding T = 1000 K even at a single point, while very small Δt values, although requiring many calculations, do not significantly increase the accuracy of the result.

This shows us once again how important the concept of efficiency, one of the most fundamental principles of engineering, is. Choosing the appropriate step interval for each problem allows us to expend the least energy for the most accurate result; otherwise, we lose either energy or accuracy.

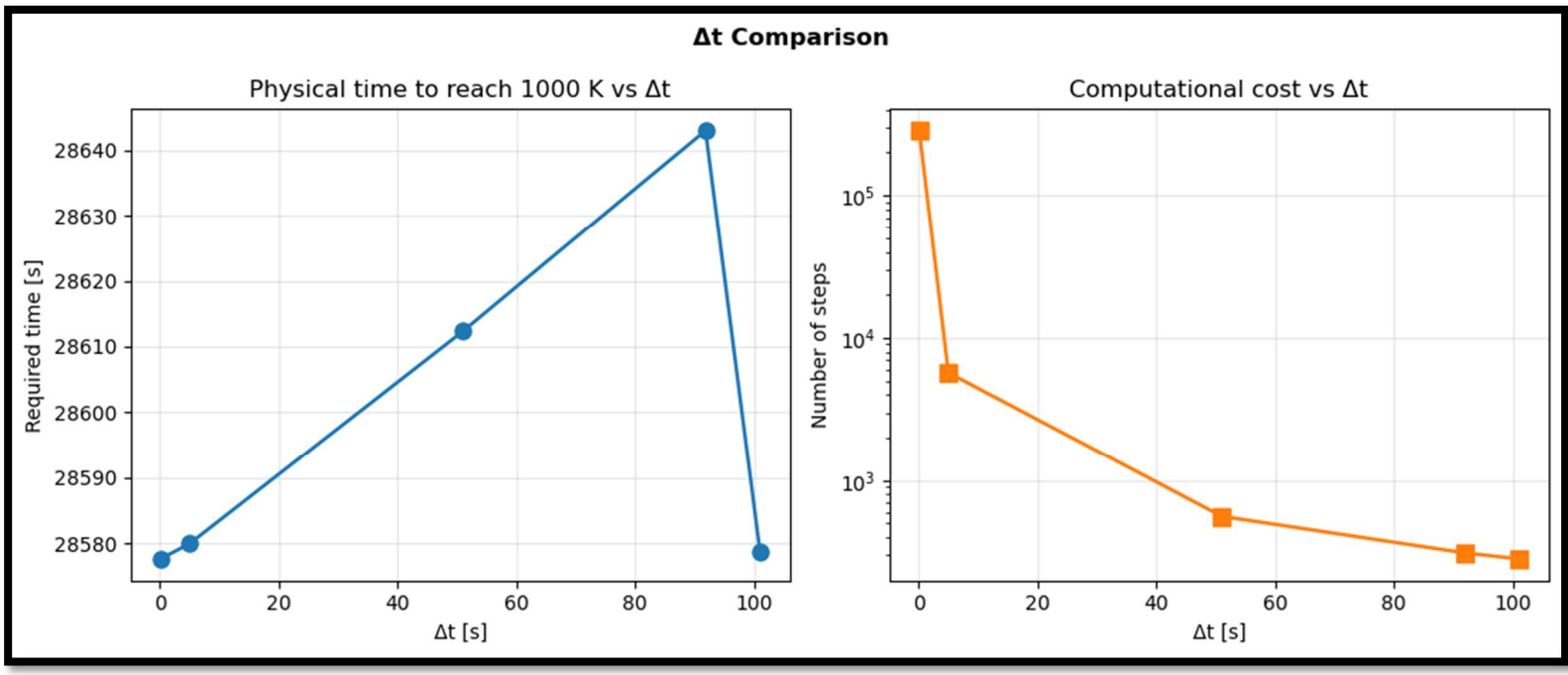


Figure 6: Comparison of total simulation time and logarithmic step count for different Δt values..

4.1.7 General Temperature Distribution

The image below shows the final temperature values obtained for each node at the end of the simulation and is color-coded according to its temperature. The hottest nodes were obtained at the nodes directly exposed to the heat source. As expected, the coldest node is the central **node (1,2) with exactly 1001.8 K.** The maximum temperature difference across the

plate is approximately 32 K, which shows that the plate exceeds the 1000 K threshold very smoothly.

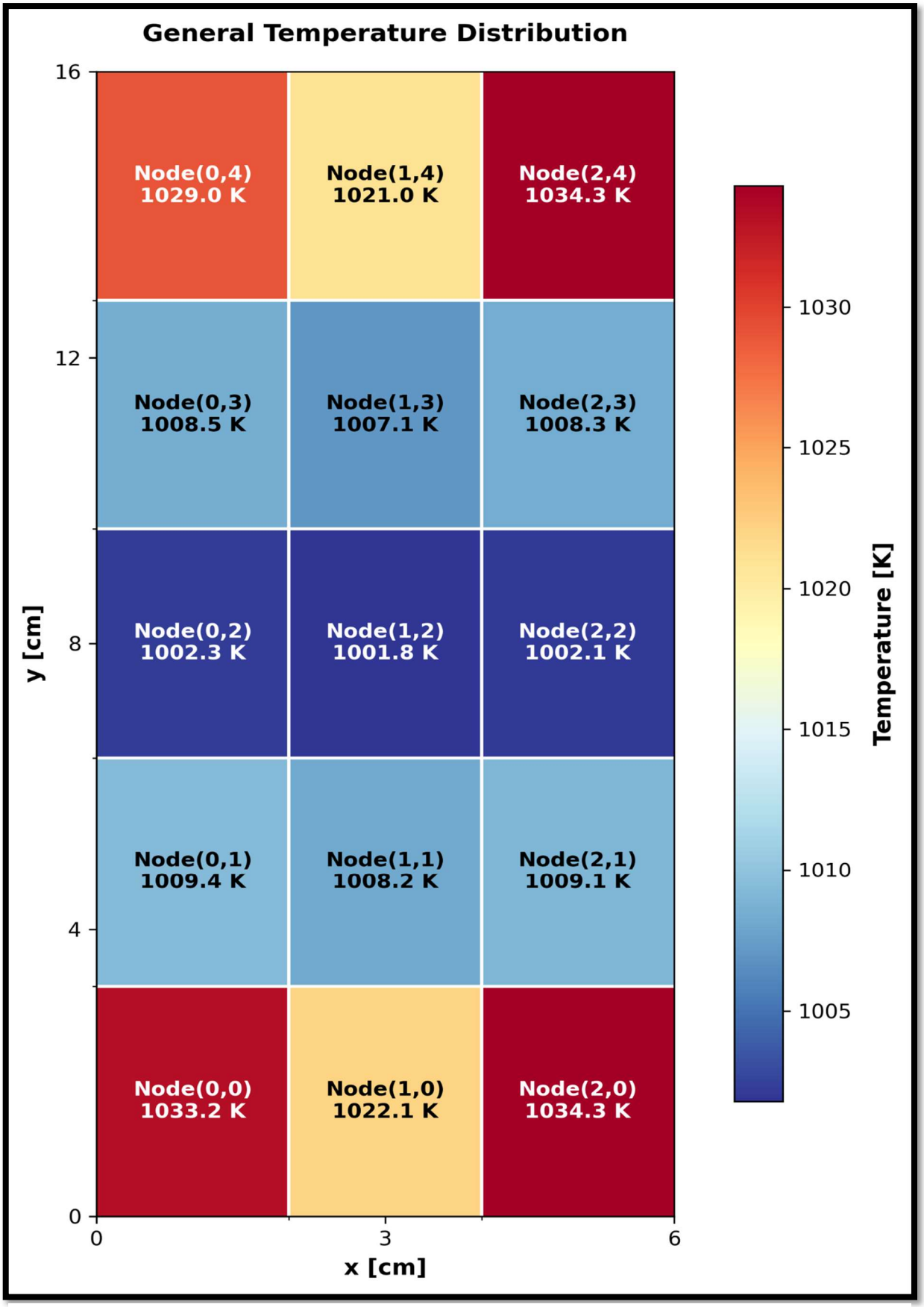


Figure 7: A 2D map showing the regional temperature for each node, along with a color scale.

4.2 Results of the Cooling Part

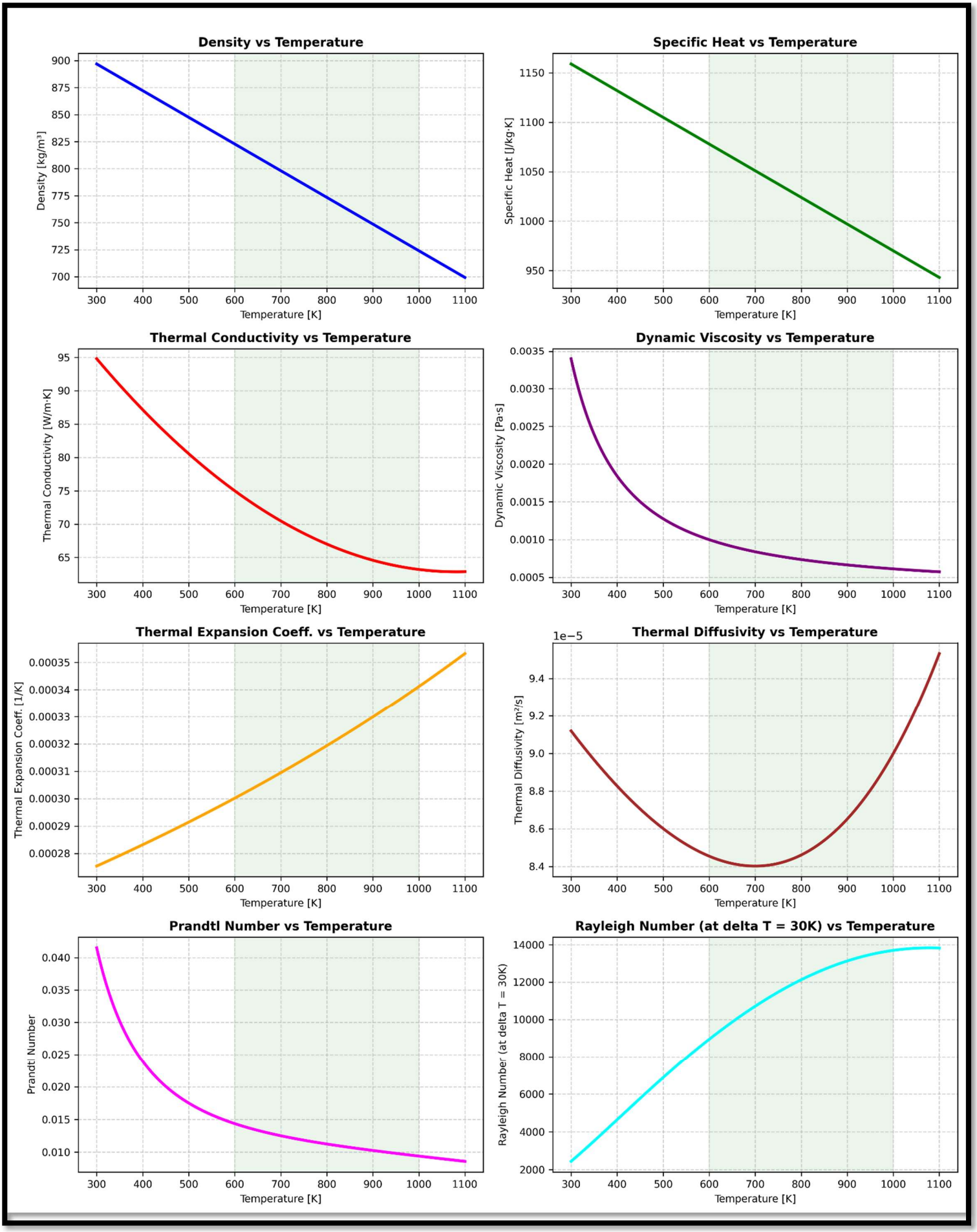


Figure 8: Temperature-dependent liquid metal coolant properties and non-dimensional hydrodynamic numbers

4.2.1 Liquid Metal Coolant Properties, Rayleigh and Prandtl Numbers

The figure above consists of eight graphs illustrating the temperature variation of critical and important thermophysical properties of liquid metal coolant. A wide temperature range was used to understand how these values change. The horizontal axis represents temperature (300–1100 K), and the vertical axes represent the respective properties. The graph also includes a green region representing the operating range of the coolant phase (600–1000 K). This part of the project is a coolant analysis, and careful reading of the graphs may be necessary.

Compared to the solid plate, the properties of the liquid metal change much more significantly with temperature. The thermal conductivity k is 95 W/m·K at 300 K and 63 W/m·K at 1000 K. Density decreases linearly in the range of 897 → 724 kg/m³, while dynamic viscosity decreases fivefold with increasing temperature, exhibiting Arrhenius-type behavior. Although the thermal expansion coefficient β is close to linear, it increases slightly faster than linear, and the Prandtl number decreases from 0.04 to 0.009. **All these situations arise depending on the temperature change. I would like to emphasize again that we need to go backward in the graph for the analysis of the cooling part.**

The information we obtained about Prandtl is important because for liquid metals**, Pr << 1** means that heat diffusion occurs much faster than momentum diffusion, and therefore the thermal boundary layer is much thicker compared to the hydrodynamic boundary layer. The Rayleigh number shown in the lower right panel was calculated for a reference temperature difference of $\Delta T = 30$ K. In the study range, the Ra value is on the order of 10^3–10^4, which confirms that natural convection occurs in the laminar regime and the validity of the correlation $Nu = 0.54\ Ra^{1/4}$.

4.2.2 Cooling Behavior of All Node Groups

The graph below shows the temperature history of the corner, edge, and inner nodes throughout the cooling phase in a single diagram. Corner nodes are shown with a red line, edge nodes with a blue line, and inner nodes with a green line; the bath temperature is represented by a black dashed line, and the target threshold of 600 K is represented by a red dotted line.

The most striking and important finding of this graph is that all three groups of nodes follow the temperature of the liquid metal bath so closely that they are indistinguishable from each other. Although there is a significant temperature difference of up to 32 Kelvin between the corner nodes (~1034 K) and inner nodes (~1002 K) at the end of the heating phase, this difference rapidly disappears in the first few seconds of the cooling phase. After that, all nodes become completely synchronized with the liquid metal bath with a very small delay of approximately 0.04 K. (It is worth noting that the process here is a controlled heat extraction process, as the temperature of the metal bath is also given as a function of time.) In short, as seen in the graph, the different nodes completely overlap.

Actually, a different and noteworthy situation arises in this part of the analysis;

Normally, the Biot number calculated for this system is quite high (Bi ≈ 31). According to classical heat transfer theory, a high Biot number predicts that the interior of the plate will not cool homogeneously, i.e., large temperature differences will occur between the center and the surface. However, we see the exact opposite in the graph, that is, the lines completely overlap and the plate stably follows the bath in terms of temperature. The most fundamental reason for this is that the bath cools very slowly. While the liquid metal bath cools slowly with a linear function, the rate at which the plate dissipates and equalizes its own heat is much higher. As a result of all these conditions, the plate is kept in a continuous, almost stable state throughout the process. In short, the cooling process is limited not by the internal resistance of the plate, but directly by the rate of temperature drop of the bath.

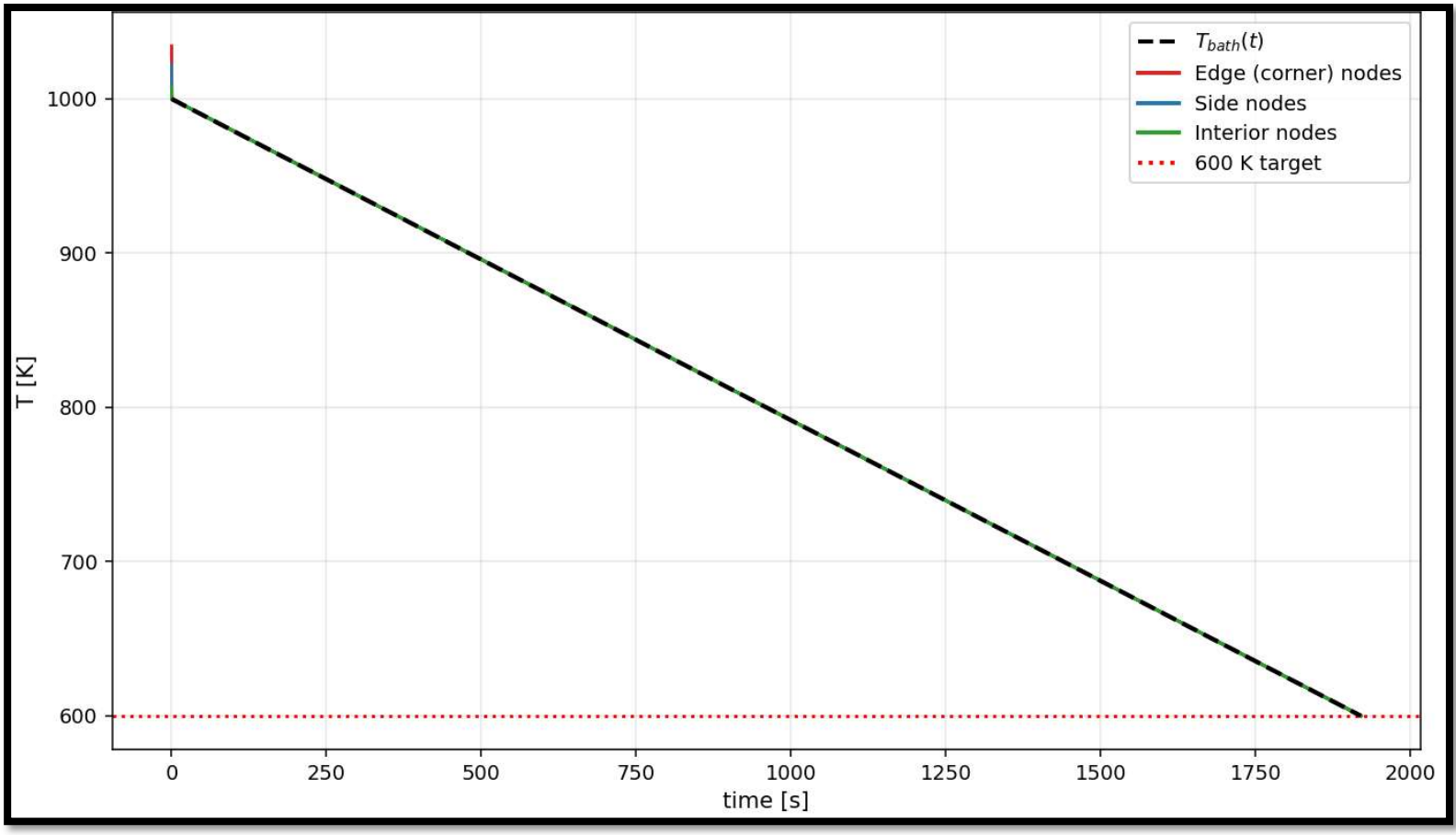


Figure 9: Temperature evolution at all node groups during cooling. Edge, side and interior nodes overlap and track the bath temperature with negligible lag.

4.2.3 Variation of Heat Transfer Coefficient Over Time

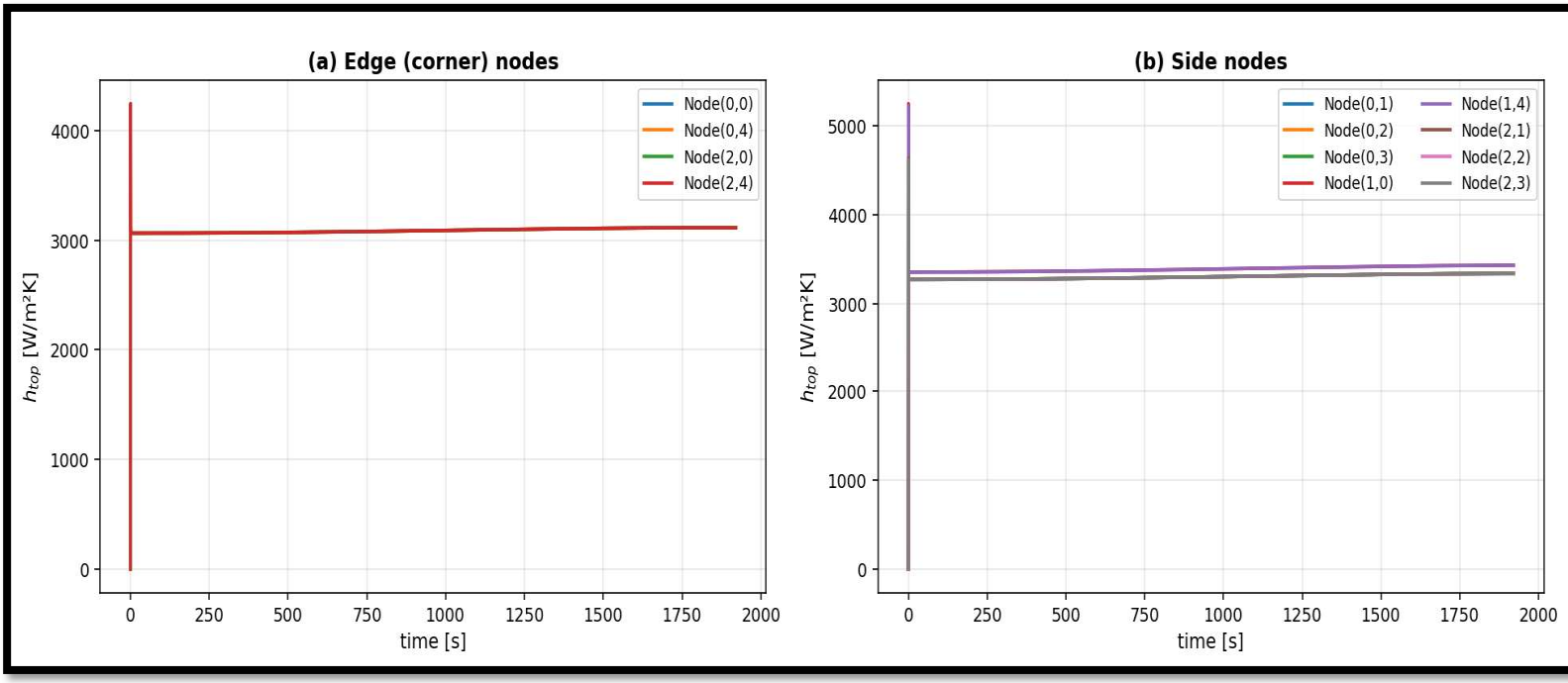


Figure 10: Time evolution of natural convection coefficient h(t) at (a) edge nodes and (b) side nodes during cooling.

A similar situation was observed at the corner and edge nodes, and this was directly reflected in the graphs. At the beginning of the simulation, the plate-bath temperature difference was in the range of 28-34 K, so the Rayleigh number had a high initial value, and h reached approximately 4500-5000 W/m²K. Subsequently, as the plate rapidly adapted to the bath, the ΔT value dropped to approximately 0.05 K. However, the h value did not approach zero, stabilizing at approximately 3100 W/m²K. The reason for this stabilization is that even at small ΔT values, as the film temperature continues to decrease, the liquid properties change significantly relative to the plate, and the Nusselt number stabilizes within the correlation structure. The h values between the corner and edge nodes are numerically very close to each other. This is an expected result, since in both groups the dominant convection surfaces are the horizontal top and bottom surfaces of the plate; the contribution of the vertical edge surfaces is limited due to the low surface area ratio.

4.2.4 The plate's temperature dropping to 600 Kelvin and an evaluation of whether this is possible.

One of the project's goals was to evaluate whether the plate nodes could cool below 600 K. The simulation results definitively answer this question: yes, all nodes can cool below 600 K, and this happens in **approximately t = 1920.3 seconds** or 32 minutes. This value is not coincidental; the time it takes for the bath temperature to drop to 600 K is also approximately

1920.3 seconds (1000 − 125·(1920/600) = 600 K), because the plate follows the bath with a very small delay of about 0.04 K and practically exceeds the 600 K threshold simultaneously with the bath. From this, we can conclude that the rate of cooling is limited not by the plate, but by the controlled rate of decrease in bath temperature. If the bath temperature could be lowered faster, the plate would also cool proportionally. This shows us that in the design of cooling processes, bath thermal management is far more critical than plate internal resistance.

4.3 Summary Table of All Results

Table 1: A table summarizing critical and important information for the project.

Parameter	Symbol / Location	Value	Unit
HEATING PHASE			
Thermal diffusivity at 1000 K	$\alpha(1000\ K)$	2.82×10^{-6}	m^2/s
Maximum allowable time step (stability)	Δt_{max}	102.0	s
Used time step	$\Delta t_{heating}$	91.8	s
Time for all nodes to reach 1000 K	$t_{heating}$	28,643 (~7.95 h)	s
Slowest-heating node (min. final T)	*Node(1,2)*	1001.8	K
Hottest final node (max. final T)	*Node(2,0)*	1034.3	K
COOLING PHASE			
Initial time step (adaptive) [1]	$\Delta t_{cooling}$	0.017	s
Time for all nodes to drop below 600 K	$t_{cooling}$	1,920.3 (~32 min)	s
Steady-state top-surface conv. coefficient	h_{top}	~3,100	W/m^2K
Final plate–bath temperature difference	ΔT_{final}	~0.04	K
DIMENSIONLESS NUMBERS			
Biot number (cooling, quasi-steady) [2]	*Bi*	≈ 31 (Bi ≫ 1)	—
Prandtl number — coolant (600–1000 K)	*Pr*	0.04 – 0.009	—
Rayleigh number — coolant (600–1000 K)	*Ra*	$10^3 – 10^4$	—

[1] Recomputed every 20 steps from the convection-dominated stability limit.
[2] Based on L_c = L_z/2 = 0.005 m, with h ≈ 3100 W/m^2K and k_s ≈ 0.5 W/mK.

4.4 ADDITIONAL ANALYSIS WITH CFD

To test the physical accuracy and numerical stability of the Finite Difference Method (FDM) code created for the project solution, a comparative time-dependent (transitional) thermal analysis was performed using Star-CCM+, a commercial Finite Volume Method (FVM) software. The computational infrastructures of the two platforms were synchronized and analyzed in terms of network structure (3x5 cell topology) and volumetric heat source definitions.

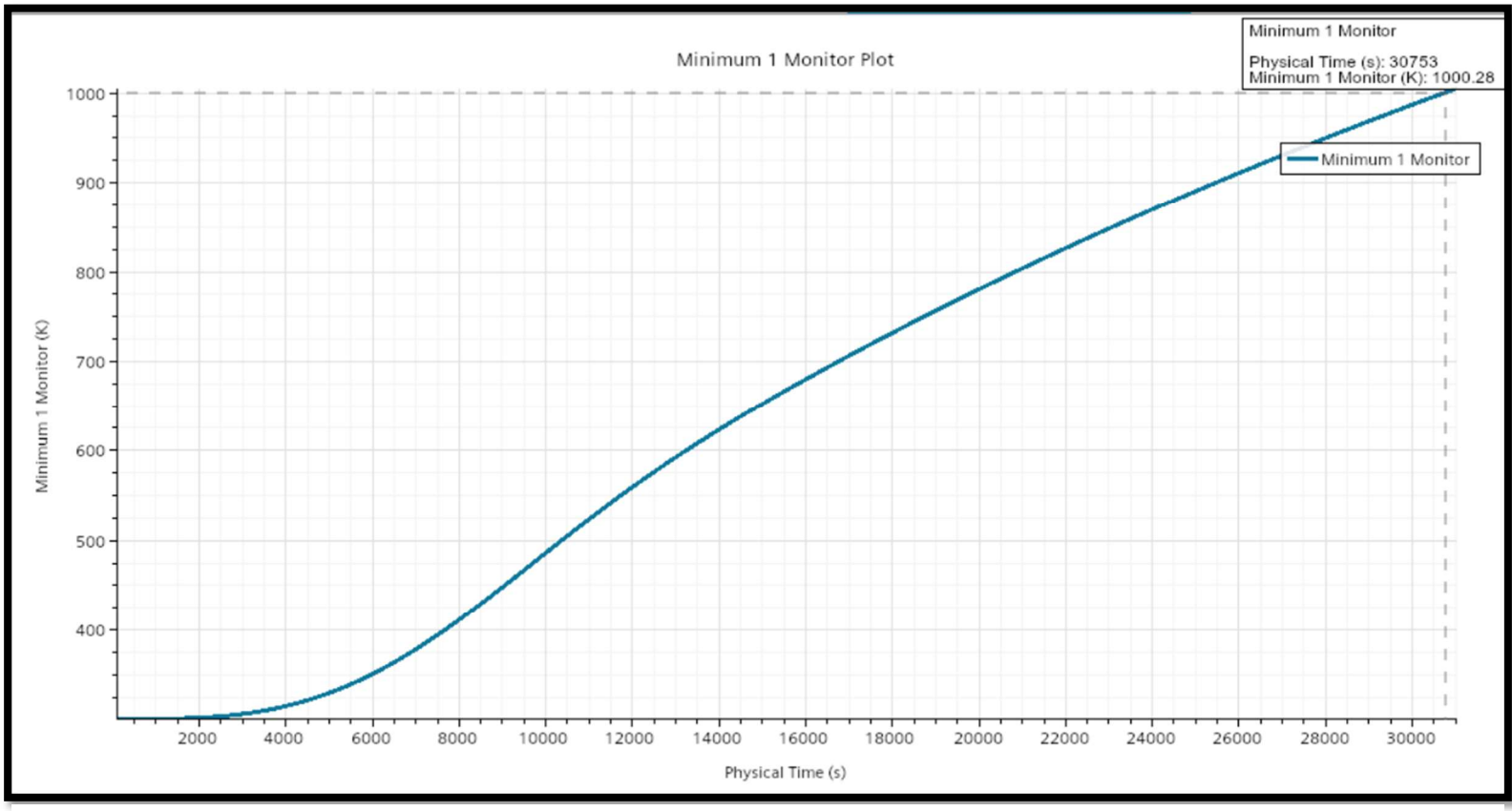


Figure 11: A graph showing the minimum temperature change of the plate over time, obtained from Star CCM+, a CFD software application.

Analysis of the heating phase reveals a significant agreement between two different numerical approaches:

Original FDM Code: 28.643 seconds (~7.95 hours)

Commercial FVM Solver (Star-CCM+): 30.753 seconds (~8.54 hours)

Relative Difference: ≈ 6.8%

For temperature-dependent material properties (k(T), ρ(T), cp(T)) and nonlinear transient simulations, the resulting 6.8% deviation represents a highly valuable and significant agreement with academic standards. The difference in time is fundamentally due to the difference in solution methods. The FDM code written for the project uses an **explicit solver,** while the CFD software uses an **implicit** matrix solver. This naturally leads to slight numerical damping and discrepancies, especially when updating properties in nonlinear temperature steps. In summary, the suitability of the chosen solution method to the problem is also one of the important factors influencing the outcome.

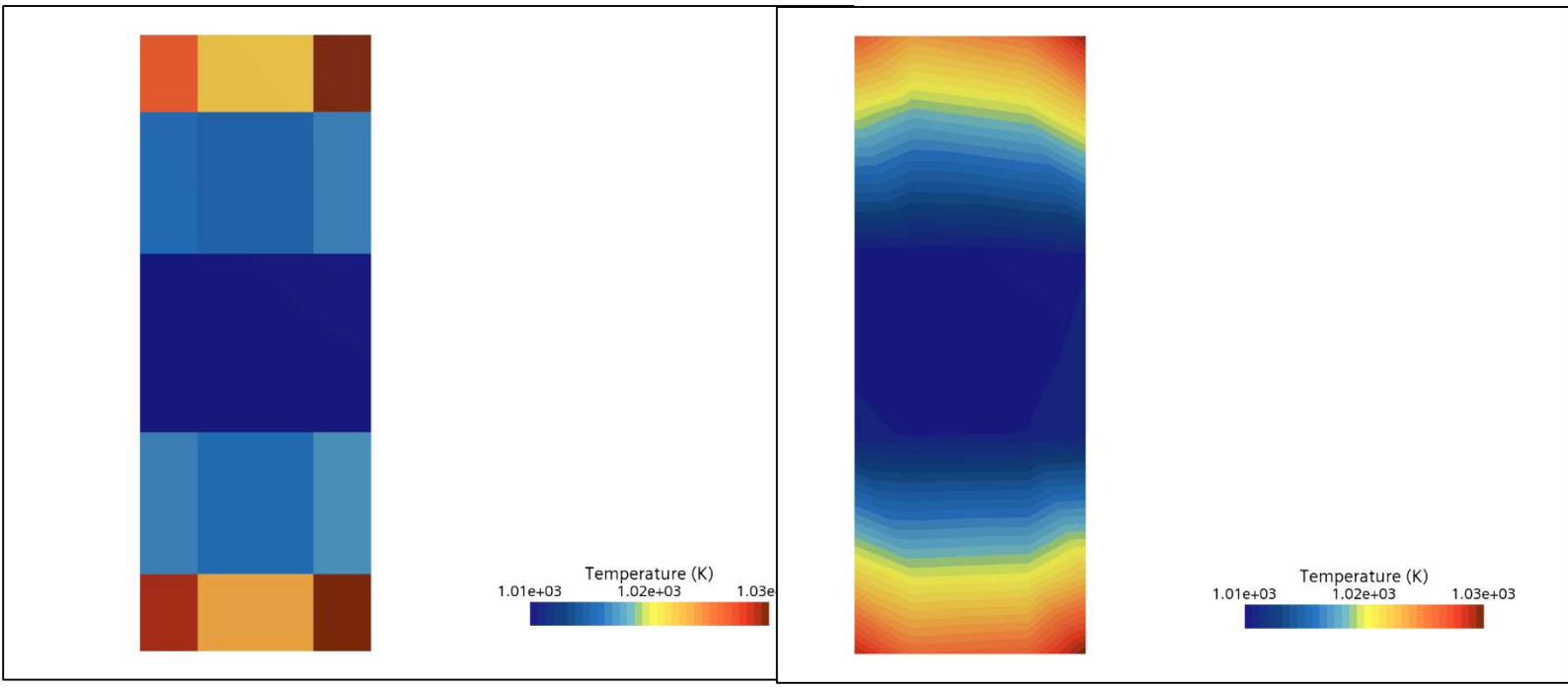


Figure 12 Mesh cells and temperature distribution in CFD application.

5 CONCLUSION

In this project, the process of heating a rectangular plate, heated at its four corners by spring clamps, with four heat sources, then removing the heat sources and cooling it in a liquid bath, was numerically investigated using the explicit finite difference method based on a control volume formulation. The properties of the plate and the liquid metal coolant w**ere not kept constant**; they were updated at each time step, depending on the instantaneous temperature values for each node; this is one of the most important factors that makes the project valuable. The findings and results obtained are summarized below.

❖ **Heating Phase and Stability Criterion:**

For the system to operate in a steady state during the heating phase, the maximum time step mathematically allowed in the explicit solution scheme was calculated as Δt_max = 102.0 s. For the project solution, a time step of **$\Delta t_heating$ = 91.8** seconds, which is 90% of this limit, was chosen and the problem was solved. The simulation showed that it takes **28643 seconds, or approximately 7.95 hours**, for all nodes to reach the 1000 K threshold. Since the heat sources are located only at the four corner nodes, the nodes in the center of the plate receive heat only indirectly through conduction; this indicates that the central node is the point that determines the total heating time.,

There is one more small detail regarding the hottest point:

Although node (2,4) had a source of 37 kW/m³, node (2,4) was not the only hottest point; node (2,0), which had a source of 35 kW/m³, also reached the same temperature. This shows us that in heat transfer, the temperature of neighboring nodes of the analyzed node is as important as the power of the source.

❖ **Time Step Comparison**

Comparisons with different time steps revealed that the physical time required to reach 1000 K was largely independent of the choice of Δt. Numerically, this time remained in the range of **28580 ± 65** seconds across all valid trials. However, the computational cost was inversely proportional to Δt: choosing Δt = 101 seconds required only 283 steps, while choosing Δt = 0.1 seconds required ($2.85777 \times 10^{5)}$ steps; that is, **1000 times more calculations** were performed to achieve the same physical result. This result demonstrates the importance of efficiency, a fundamental principle of engineering, regardless of the problem. It is clear how important a Δt choice that is close to the stability limit but safe is, both in terms of accuracy and computational efficiency.

❖ **Liquid Metal Cooling Performance**

In the second phase of the project, the heat sources on the plate were removed, and then the plate was immersed in a liquid metal cooling bath whose temperature decreased with a time-dependent linear ramp function. Subsequently, the natural convection boundary condition based on the Rayleigh number was applied. **It took 1920.3 seconds (~32 minutes)** for all nodes to reach below the target temperature of 600 K. The ultra-low Prandtl number of the liquid metals ($Pr \leq 1$, $0.04 \approx 0.009$) indicated that molecular heat diffusion completely dominated momentum diffusion, resulting in the formation of a very thick thermal boundary

layer on the plate surface. This thermal characteristic allowed for the achievement of a very high convection coefficient htop ≈ 3100 W/m² K in the steady state, and the plate-bath temperature difference was stabilized at ~0.04 K.

❖ The Biot Number Paradox and Quasi-Stable Regimes

The calculated Biot number during the cooling phase is Bi ≈ 31, satisfying the condition Bi >> 1. Classical heat transfer theory predicts that significant temperature gradients will form within the plate in this case. However, simulation results show that all nodal lines visually coincide with the bath temperature. The reason for this seemingly contradictory situation is as follows: While the internal thermal equilibrium time of the plate is $\tau_{internal} \approx L^2/\alpha_s \approx 319$ s, the cooling time scale of the bath is $\tau_{bath} \approx 1920$ s. Therefore, since $\tau_{bath} >> \tau_{internal}$, the plate reaches its internal equilibrium before the bath temperature changes significantly at each step. In summary, in this unsteady regime, the instantaneous spatial temperature difference across the plate remains only on the order of ~0.62 K and becomes indistinguishable on the graphical scale. Thus, this observed behavior is due not to the absence of internal resistance, but to the decomposition of time scales.

❖ Natural Convection Regime

The fact that the Rayleigh number of the coolant in the project is on the order of 10^3 – 10^4 in the operating range (600–1000 K) confirms that natural convection remains in a completely laminar regime, and correlations have been established accordingly; for example, this clearly makes the correlation Nu = 0.54 Ra^(1/4) usable. The very low Prandtl number of the liquid metal (Pr = 0.009 → 0.04 as it gradually cools) led to thermal diffusion dominating momentum diffusion and the formation of a thick thermal boundary layer on the surface; this formed the physical basis for the high h values in the project.

❖ An important conclusion drawn from the results obtained is:

The most critical finding of this project is that in liquid metal-cooled systems with high heat transfer capacity, the transient cooling rate is limited not by the internal conduction resistance of the solid component, but **entirely by the thermal management and ramp rate of the refrigerant bath** (bath-controlled cooling). In other words, if the cooling rate of the bath is slowed down, the plate cools proportionally slower; if it is accelerated, the plate cools proportionally faster. This finding clearly demonstrates that controlling the rate of fluid temperature decrease is the most decisive and critical parameter in cooling processes and the design of thermal control systems.

6 REFERENCES

[1] T. L. Bergman, A. S. Lavine, F. P. Incropera, and D. P. DeWitt, *Fundamentals of Heat and Mass Transfer*, 7th ed. Hoboken, NJ: John Wiley & Sons, 2011.

[2] Y. A. Cengel and A. J. Ghajar, *Heat and Mass Transfer: Fundamentals and Applications*, 5th ed. New York, NY: McGraw-Hill Education, 2015.

[3] S. W. Churchill and H. H. S. Chu, "Correlating equations for laminar and turbulent free convection from a vertical plate," *International Journal of Heat and Mass Transfer*, vol. 18, no. 11, pp. 1323–1329, 1975, doi: 10.1016/0017-9310(75)90243-4.

[4] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*. Washington, DC: Hemisphere Publishing Corporation, 1980.

[5] M. N. Özışık, *Heat Conduction*, 2nd ed. New York, NY: John Wiley & Sons, 1993.

[6] J. K. Fink and L. Leibowitz, *Thermophysical Properties of Sodium*, Argonne National Laboratory, Report ANL/RE-95/2, 1995.

7 APPENDIX

```
# 1: Localized volumectric heating
# 2: Natural convection cooling within a liquid metal bath

import numpy as np
import matplotlib.pyplot as plt



# Solid plate properties (temperature dependent!!!)

def solid_conductivity(T):
    k0 = 0.0025
    a  = 5.0e-6
    b  = 5.0e-10
    k = k0 + a * T + b * T * T * T
    return k

def solid_specific_heat(T):
```

```
    cp = 700.0 + 0.2 * T
    return cp

def solid_density(T):
    rho0 = 200.0
    beta = 2.0e-6
    rho = rho0 * (1.0 - beta * (T - 300.0))
    return rho

def solid_diffusivity(T):
    # alpha = k / (rho * cp)
    k  = solid_conductivity(T)
    cp = solid_specific_heat(T)
    rho = solid_density(T)
    alpha = k / (rho * cp)
    return alpha


# Liquid metal coolant properties (temperature dependent)
#
def liquid_conductivity(T):
    k = 124.0 - 0.113 * T + 5.22e-5 * T * T
    return k

def liquid_density(T):
    rho = 971.0 - 0.247 * T
    return rho

def liquid_expansion(T):
    beta = 0.247 / (971.0 - 0.247 * T)
    return beta

def liquid_viscosity(T):
    mu = 2.94e-4 * np.exp(734.0 / T)
    return mu

def liquid_specific_heat(T):
    cp = 1240.0 - 0.27 * T
    return cp
```

```
def liquid_kinematic_viscosity(T):
    mu  = liquid_viscosity(T)
    rho = liquid_density(T)
    nu = mu / rho
    return nu

def liquid_diffusivity(T):
    k   = liquid_conductivity(T)
    rho = liquid_density(T)
    cp  = liquid_specific_heat(T)
    alpha = k / (rho * cp)
    return alpha

def liquid_prandtl(T):
    mu = liquid_viscosity(T)
    cp = liquid_specific_heat(T)
    k  = liquid_conductivity(T)
    Pr = mu * cp / k
    return Pr


def bath_temperature(t):
    # The bath cools down linearly with time.
    Tb = 1000.0 - 125.0 * (t / 600.0)
    return Tb


# Geometry and grid
Lx = 0.06     # plate width  [m]
Ly = 0.16     # plate length [m]
Lz = 0.01     # plate thickness [m]

dx = 0.03     # node spacing in x [m]
dy = 0.04     # node spacing in y [m]

nx = 3        # number of nodes in x  ( Lx/dx + 1 )
ny = 5        # number of nodes in y  ( Ly/dy + 1 )

g = 9.81      # gravity [m/s^2]
```

```
    # Characteristic lengths used in the convection correlations.
    # Horizontal faces use the standard As/P definition.
    Lc_horizontal = (Lx * Ly) / (2.0 * (Lx + Ly))
    Lc_vertical   = Lz

    # Boundary nodes only own half of a cell
    def cv_width_x(i):
        if i == 0 or i == nx - 1:
            return dx / 2.0
        else:
            return dx

    def cv_width_y(j):
        if j == 0 or j == ny - 1:
            return dy / 2.0
        else:
            return dy



    # Heat generation: only the four corner nodes have a source.
    # Other nodes does not have a heat generation

    qgen = np.zeros((nx, ny))
    qgen[0,      0]      = 32000.0   # bottom-left clamp
    qgen[nx - 1, 0]      = 35000.0   # bottom-right clamp
    qgen[0,      ny - 1] = 28000.0   # top-left clamp
    qgen[nx - 1, ny - 1] = 37000.0   # top-right clamp



    #  ------------------------------------------------------------------
----------
    # Classify the nodes into three groups so I can plot them separately
later.
    # I just walk over the whole grid and decide group by position.
    #  ------------------------------------------------------------------
----------
    corner_nodes   = []
    side_nodes     = []
    interior_nodes = []

    for i in range(0, nx):
```

```
        for j in range(0, ny):
            is_corner = (i == 0 or i == nx - 1) and (j == 0 or j == ny -
1)
            is_edge   = (i == 0 or i == nx - 1 or j == 0 or j == ny - 1)
            if is_corner:
                corner_nodes.append((i, j))
            elif is_edge:
                side_nodes.append((i, j))
            else:
                interior_nodes.append((i, j))



    #
    # PART A - Stability criterion
    # The most restrictive case is at the highest temperature (1000 K)
    # the diffusivity increases strongly with temperature bcs (cubic term
in k).
    alpha_at_1000 = solid_diffusivity(1000.0)
    dt_max = 1.0 / (2.0 * alpha_at_1000 * (1.0 / (dx * dx) + 1.0 / (dy *
dy)))

    print("---------- PART A: Stability ----------")
    print("alpha(300 K)  =", solid_diffusivity(300.0), "m^2/s")
    print("alpha(1000 K) =", alpha_at_1000, "m^2/s")
    print("dt_max        =", round(dt_max, 3), "s")

    # Use 90% of the limit for the main heating run.
    dt_heat = 0.90 * dt_max
    print("dt_heat used  =", round(dt_heat, 3), "s")



    # PART B - Heating simulation
    # Boundaries are adiabatic, so a missing neighbor simply contributes
no flux.
    # I build the new temperature field node by node.
    print("\n---------- PART B: Heating ----------")

    T = np.zeros((nx, ny))
    for i in range(0, nx):
        for j in range(0, ny):
            T[i, j] = 300.0     # initial temperature everywhere
```

```
    time_now = 0.0
    heat_time_list = [0.0]
    heat_T_history = [T.copy()]

    # Keep stepping until even the coldest node has reached 1000 K.
    keep_going = True
    while keep_going:

        T_new = T.copy()

        for i in range(0, nx):
            for j in range(0, ny):

                T_here = T[i, j]
                k_here = solid_conductivity(T_here)

                # control volume of this node
                wx = cv_width_x(i)
                wy = cv_width_y(j)
                volume = wx * wy * Lz

                # collect conduction from each existing neighbor
                Q_cond = 0.0

                # neighbor in +x
                if i < nx - 1:
                    k_face = 0.5 * (k_here + solid_conductivity(T[i + 1,
j]))
                    area   = wy * Lz
                    Q_cond = Q_cond + k_face * area * (T[i + 1, j] -
T_here) / dx

                # neighbor in -x
                if i > 0:
                    k_face = 0.5 * (k_here + solid_conductivity(T[i - 1,
j]))
                    area   = wy * Lz
                    Q_cond = Q_cond + k_face * area * (T[i - 1, j] -
T_here) / dx

                # neighbor in +y
                if j < ny - 1:
```

```
                    k_face = 0.5 * (k_here + solid_conductivity(T[i, j +
1]))
                    area   = wx * Lz
                    Q_cond = Q_cond + k_face * area * (T[i, j + 1] -
T_here) / dy

                # neighbor in -y
                if j > 0:
                    k_face = 0.5 * (k_here + solid_conductivity(T[i, j -
1]))
                    area   = wx * Lz
                    Q_cond = Q_cond + k_face * area * (T[i, j - 1] -
T_here) / dy

                # internal heat generation of this node
                Q_source = qgen[i, j] * volume

                # update this node
                rho_here = solid_density(T_here)
                cp_here  = solid_specific_heat(T_here)
                heat_capacity = rho_here * cp_here * volume

                T_new[i, j] = T_here + dt_heat * (Q_cond + Q_source) /
heat_capacity

        T = T_new
        time_now = time_now + dt_heat

        heat_time_list.append(time_now)
        heat_T_history.append(T.copy())

        # find the minimum temperature on the grid by hand
        T_min_now = T[0, 0]
        for i in range(0, nx):
            for j in range(0, ny):
                if T[i, j] < T_min_now:
                    T_min_now = T[i, j]

        if T_min_now >= 1000.0:
            keep_going = False

    heat_time = np.array(heat_time_list)
```

```
    heat_T_history = np.array(heat_T_history)
    T_after_heating = T.copy()

    # locate the slowest and the hottest node
    slow_i, slow_j = 0, 0
    hot_i, hot_j   = 0, 0
    for i in range(0, nx):
        for j in range(0, ny):
            if T[i, j] < T[slow_i, slow_j]:
                slow_i, slow_j = i, j
            if T[i, j] > T[hot_i, hot_j]:
                hot_i, hot_j = i, j

    print("Heating finished at t =", round(time_now, 1), "s (",
round(time_now / 3600.0, 2), "hours )")
    print("Slowest node =", (slow_i, slow_j), " T =", round(T[slow_i,
slow_j], 2), "K")
    print("Hottest node =", (hot_i, hot_j), " T =", round(T[hot_i,
hot_j], 2), "K")



    # PART C - Effect of the time step
    # I repeat the heating run for several dt values and record how long
it takes
    # and how many steps are needed
    print("\n---------- PART C: dt comparison ----------")

    dt_try_list = [0.99 * dt_max, 0.90 * dt_max, 0.50 * dt_max, 5.0, 0.1]
    partc_dt    = []
    partc_time  = []
    partc_steps = []

    for dt_try in dt_try_list:

        Tc = np.zeros((nx, ny))
        for i in range(0, nx):
            for j in range(0, ny):
                Tc[i, j] = 300.0

        tt = 0.0
        nstep = 0
```

```
        diverged = False

        busy = True
        while busy:

            Tc_new = Tc.copy()
            for i in range(0, nx):
                for j in range(0, ny):
                    T_here = Tc[i, j]
                    k_here = solid_conductivity(T_here)
                    wx = cv_width_x(i)
                    wy = cv_width_y(j)
                    volume = wx * wy * Lz
                    Q_cond = 0.0
                    if i < nx - 1:
                        k_face = 0.5 * (k_here + solid_conductivity(Tc[i +
1, j]))
                        Q_cond = Q_cond + k_face * (wy * Lz) * (Tc[i + 1,
j] - T_here) / dx
                    if i > 0:
                        k_face = 0.5 * (k_here + solid_conductivity(Tc[i -
1, j]))
                        Q_cond = Q_cond + k_face * (wy * Lz) * (Tc[i - 1,
j] - T_here) / dx
                    if j < ny - 1:
                        k_face = 0.5 * (k_here + solid_conductivity(Tc[i,
j + 1]))
                        Q_cond = Q_cond + k_face * (wx * Lz) * (Tc[i, j +
1] - T_here) / dy
                    if j > 0:
                        k_face = 0.5 * (k_here + solid_conductivity(Tc[i,
j - 1]))
                        Q_cond = Q_cond + k_face * (wx * Lz) * (Tc[i, j -
1] - T_here) / dy
                    Q_source = qgen[i, j] * volume
                    heat_capacity      =      solid_density(T_here)      *
solid_specific_heat(T_here) * volume
                    Tc_new[i, j] = T_here + dt_try * (Q_cond + Q_source) /
heat_capacity

            Tc = Tc_new
            tt = tt + dt_try
            nstep = nstep + 1
```

```
            # blow-up check
            if (not np.all(np.isfinite(Tc))) or (Tc.max() > 1.0e6):
                diverged = True
                busy = False

            # stop condition
            Tc_min = Tc.min()
            if Tc_min >= 1000.0:
                busy = False

            # hard safety cap so the loop never runs forever
            if nstep > 5000000:
                busy = False

        if diverged:
            print("dt =", round(dt_try, 3), "s  -> DIVERGED")
        else:
            print("dt =", round(dt_try, 3), "s  -> t =", round(tt, 1),
                  "s ,", nstep, "steps")
            partc_dt.append(dt_try)
            partc_time.append(tt)
            partc_steps.append(nstep)



    # PART D - Cooling in the liquid metal bath
    # There is no heat generation now. Every face of the plate loses heat by natural
    # convection to the bath. h is computed locally with the Rayleigh number.
    print("\n---------- PART D: Cooling ----------")


    # This helper returns h for one face. I keep the three orientations explicit.
    def convection_coefficient(T_surface, T_bath, Lc, orientation):

        delta_T = abs(T_surface - T_bath)
        if delta_T < 1.0e-3:
            return 0.0

        T_film = 0.5 * (T_surface + T_bath)
        if T_film < 50.0:
```

```
            T_film = 50.0

        beta  = liquid_expansion(T_film)
        nu    = liquid_kinematic_viscosity(T_film)
        alpha = liquid_diffusivity(T_film)
        k     = liquid_conductivity(T_film)
        Pr    = liquid_prandtl(T_film)

        Ra = g * beta * delta_T * Lc * Lc * Lc / (nu * alpha)
        if Ra <= 0.0:
            return 0.0

        if orientation == "top":
            if Ra < 1.0e7:
                Nu = 0.54 * Ra ** 0.25
            else:
                Nu = 0.15 * Ra ** (1.0 / 3.0)
        elif orientation == "bottom":
            Nu = 0.27 * Ra ** 0.25
        else:
            # vertical side faces, Churchill-Chu
            denom = (1.0 + (0.492 / Pr) ** (9.0 / 16.0)) ** (4.0 / 9.0)
            Nu = 0.68 + 0.670 * Ra ** 0.25 / denom

        h = Nu * k / Lc
        return h



    # To stay stable I recompute the allowed step from the current field.
    # The convection term dominates, so I look at h*A/(rho*cp*V) plus
conduction.
    def cooling_stable_dt(T_field, t_value):

        Tb = bath_temperature(t_value)
        worst = 0.0

        for i in range(0, nx):
            for j in range(0, ny):

                T_here = T_field[i, j]
```

```
                wx = cv_width_x(i)
                wy = cv_width_y(j)
                volume = wx * wy * Lz
                rcpV = solid_density(T_here) * solid_specific_heat(T_here)
* volume

                face_area = wx * wy
                h_top = convection_coefficient(T_here, Tb, Lc_horizontal,
"top")
                h_bot = convection_coefficient(T_here, Tb, Lc_horizontal,
"bottom")
                hA = (h_top + h_bot) * face_area

                if i == 0 or i == nx - 1:
                    hv = convection_coefficient(T_here, Tb, Lc_vertical,
"vert")
                    hA = hA + hv * wy * Lz
                if j == 0 or j == ny - 1:
                    hv = convection_coefficient(T_here, Tb, Lc_vertical,
"vert")
                    hA = hA + hv * wx * Lz

                cond_part = 2.0 * solid_diffusivity(T_here) * (1.0 / (dx *
dx) + 1.0 / (dy * dy))
                value = hA / rcpV + cond_part
                if value > worst:
                    worst = value

        if worst > 0.0:
            return 1.0 / worst
        else:
            return 1.0


    # start cooling from the heated field
    T = T_after_heating.copy()

    dt_cool = 0.40 * cooling_stable_dt(T, 0.0)
    print("initial cooling dt =", round(dt_cool, 4), "s")

    cool_time_list = [0.0]
    cool_T_history = [T.copy()]
```

```
    htop_history   = [np.zeros((nx, ny))]
    bath_history   = [bath_temperature(0.0)]

    t_cool = 0.0
    step_count = 0
    save_every = 25

    cooling = True
    while cooling:

        # refresh the stable step every once in a while
        if step_count % 20 == 0:
            candidate = 0.40 * cooling_stable_dt(T, t_cool)
            if candidate < dt_cool:
                dt_cool = candidate

        Tb = bath_temperature(t_cool)
        T_new = T.copy()
        htop_now = np.zeros((nx, ny))

        for i in range(0, nx):
            for j in range(0, ny):

                T_here = T[i, j]
                k_here = solid_conductivity(T_here)
                wx = cv_width_x(i)
                wy = cv_width_y(j)
                volume = wx * wy * Lz

                # conduction from neighbors (same as heating)
                Q_cond = 0.0
                if i < nx - 1:
                    k_face = 0.5 * (k_here + solid_conductivity(T[i + 1,
j]))
                    Q_cond = Q_cond + k_face * (wy * Lz) * (T[i + 1, j] -
T_here) / dx
                if i > 0:
                    k_face = 0.5 * (k_here + solid_conductivity(T[i - 1,
j]))
                    Q_cond = Q_cond + k_face * (wy * Lz) * (T[i - 1, j] -
T_here) / dx
                if j < ny - 1:
```

```
                    k_face = 0.5 * (k_here + solid_conductivity(T[i, j +
1]))
                    Q_cond = Q_cond + k_face * (wx * Lz) * (T[i, j + 1] -
T_here) / dy
                if j > 0:
                    k_face = 0.5 * (k_here + solid_conductivity(T[i, j -
1]))
                    Q_cond = Q_cond + k_face * (wx * Lz) * (T[i, j - 1] -
T_here) / dy

                # convection on top and bottom faces
                face_area = wx * wy
                h_top = convection_coefficient(T_here, Tb, Lc_horizontal,
"top")
                h_bot = convection_coefficient(T_here, Tb, Lc_horizontal,
"bottom")
                hA = (h_top + h_bot) * face_area

                # convection on the outer vertical faces (boundary nodes
only)
                if i == 0 or i == nx - 1:
                    hv = convection_coefficient(T_here, Tb, Lc_vertical,
"vert")
                    hA = hA + hv * wy * Lz
                if j == 0 or j == ny - 1:
                    hv = convection_coefficient(T_here, Tb, Lc_vertical,
"vert")
                    hA = hA + hv * wx * Lz

                Q_conv = - hA * (T_here - Tb)

                heat_capacity = solid_density(T_here) *
solid_specific_heat(T_here) * volume
                T_new[i, j] = T_here + dt_cool * (Q_cond + Q_conv) /
heat_capacity

                htop_now[i, j] = h_top

        T = T_new
        t_cool = t_cool + dt_cool
        step_count = step_count + 1

        if step_count % save_every == 0:
            cool_time_list.append(t_cool)
```

```
            cool_T_history.append(T.copy())
            htop_history.append(htop_now.copy())
            bath_history.append(Tb)

        # stop once the hottest node is below 600 K
        T_max_now = T[0, 0]
        for i in range(0, nx):
            for j in range(0, ny):
                if T[i, j] > T_max_now:
                    T_max_now = T[i, j]

        if T_max_now <= 600.0:
            cooling = False

    cool_time = np.array(cool_time_list)
    cool_T_history = np.array(cool_T_history)
    htop_history = np.array(htop_history)
    bath_history = np.array(bath_history)

    print("Cooling finished at t =", round(t_cool, 2), "s (",
round(t_cool / 60.0, 2), "min )")
    print("final T_max =", round(T.max(), 2), "K , bath =",
round(bath_temperature(t_cool), 2), "K")



    # ---------------------------------
    # FIGURES

    # ---- Figure 1: solid properties ----
    T_axis = np.linspace(300.0, 1100.0, 300)
    k_curve  = np.zeros(len(T_axis))
    rho_curve = np.zeros(len(T_axis))
    cp_curve = np.zeros(len(T_axis))
    for n in range(0, len(T_axis)):
        k_curve[n]   = solid_conductivity(T_axis[n])
        rho_curve[n] = solid_density(T_axis[n])
        cp_curve[n]  = solid_specific_heat(T_axis[n])

    fig1, ax1 = plt.subplots(1, 3, figsize=(13, 4))
    ax1[0].plot(T_axis, k_curve, color="red")
    ax1[0].set_xlabel("T [K]"); ax1[0].set_ylabel("k_s [W/m K]")
```

```
    ax1[0].set_title("Conductivity"); ax1[0].grid(alpha=0.3)
    ax1[1].plot(T_axis, rho_curve, color="blue")
    ax1[1].set_xlabel("T [K]"); ax1[1].set_ylabel("rho_s [kg/m3]")
    ax1[1].set_title("Density"); ax1[1].grid(alpha=0.3)
    ax1[2].plot(T_axis, cp_curve, color="green")
    ax1[2].set_xlabel("T [K]"); ax1[2].set_ylabel("cp_s [J/kg K]")
    ax1[2].set_title("Specific Heat"); ax1[2].grid(alpha=0.3)
    fig1.suptitle("Figure 1 - Solid Plate Properties")
    fig1.tight_layout()
    fig1.savefig("fig01_solid_props.png", dpi=130)

    # ---- Figure 2,3,4: heating temperature for each group ----
    def plot_heating_group(node_list, title_text, file_name):
        fig, ax = plt.subplots(figsize=(9, 5))
        for (i, j) in node_list:
            if qgen[i, j] > 0.0:
                label_text = "Node(" + str(i) + "," + str(j) + ")  q=" +
str(int(qgen[i, j] / 1000.0)) + " kW/m3"
            else:
                label_text = "Node(" + str(i) + "," + str(j) + ")"
            ax.plot(heat_time,  heat_T_history[:,  i,  j],  linewidth=1.8,
label=label_text)
        ax.axhline(1000.0,  color="black",  linestyle="--",  linewidth=1.2,
label="1000 K target")
        ax.set_xlabel("time [s]"); ax.set_ylabel("T [K]")
        ax.set_title(title_text);                        ax.grid(alpha=0.3);
ax.legend(fontsize=9)
        fig.tight_layout()
        fig.savefig(file_name, dpi=130)

    plot_heating_group(corner_nodes,    "Figure 2 - Heating: Edge (Corner)
Nodes", "fig02_heat_edge.png")
    plot_heating_group(side_nodes,      "Figure 3 - Heating: Side Nodes",
"fig03_heat_side.png")
    plot_heating_group(interior_nodes,  "Figure  4  -  Heating:  Interior
Nodes",      "fig04_heat_interior.png")

    # ---- Figure 5: dt comparison (Part C) ----
    fig5, ax5 = plt.subplots(1, 2, figsize=(11, 4.5))
    ax5[0].plot(partc_dt, partc_time, marker="o")
    ax5[0].set_xlabel("dt [s]"); ax5[0].set_ylabel("required time [s]")
    ax5[0].set_title("Time to reach 1000 K"); ax5[0].grid(alpha=0.3)
    ax5[1].plot(partc_dt, partc_steps, marker="s", color="orange")
```

```
    ax5[1].set_yscale("log")
    ax5[1].set_xlabel("dt [s]"); ax5[1].set_ylabel("number of steps")
    ax5[1].set_title("Computational cost"); ax5[1].grid(alpha=0.3)
    fig5.suptitle("Figure 5 - Effect of Time Step (Part C)")
    fig5.tight_layout()
    fig5.savefig("fig05_dt_comparison.png", dpi=130)

    # ---- Figure 6: liquid properties + Rayleigh ----
    Tl_axis = np.linspace(300.0, 1100.0, 300)
    kl_c  = np.zeros(len(Tl_axis))
    rhol_c = np.zeros(len(Tl_axis))
    cpl_c = np.zeros(len(Tl_axis))
    mul_c = np.zeros(len(Tl_axis))
    betal_c = np.zeros(len(Tl_axis))
    Ra_c  = np.zeros(len(Tl_axis))
    for n in range(0, len(Tl_axis)):
        Tn = Tl_axis[n]
        kl_c[n]   = liquid_conductivity(Tn)
        rhol_c[n] = liquid_density(Tn)
        cpl_c[n]  = liquid_specific_heat(Tn)
        mul_c[n]  = liquid_viscosity(Tn)
        betal_c[n] = liquid_expansion(Tn)
        if Tn > 330.0:
            Ra_c[n] = g * liquid_expansion(Tn) * 30.0 * Lc_horizontal ** 3
/ (liquid_kinematic_viscosity(Tn) * liquid_diffusivity(Tn))
        else:
            Ra_c[n] = 0.0

    fig6, ax6 = plt.subplots(2, 3, figsize=(14, 8))
    ax6[0,    0].plot(Tl_axis,    kl_c,    color="red");              ax6[0,
0].set_title("Conductivity");  ax6[0, 0].set_ylabel("k_l [W/m K]")
    ax6[0,     1].plot(Tl_axis,     rhol_c,     color="blue");      ax6[0,
1].set_title("Density");       ax6[0, 1].set_ylabel("rho_l [kg/m3]")
    ax6[0,     2].plot(Tl_axis,     cpl_c,     color="green");      ax6[0,
2].set_title("Specific Heat"); ax6[0, 2].set_ylabel("cp_l [J/kg K]")
    ax6[1,     0].plot(Tl_axis,     mul_c,     color="purple");     ax6[1,
0].set_title("Viscosity");     ax6[1, 0].set_ylabel("mu_l [Pa s]"); ax6[1,
0].set_yscale("log")
    ax6[1,     1].plot(Tl_axis,     betal_c,     color="brown");     ax6[1,
1].set_title("Expansion");   ax6[1, 1].set_ylabel("beta_l [1/K]")
    ax6[1,    2].plot(Tl_axis,    Ra_c,    color="black");          ax6[1,
2].set_title("Rayleigh   (dT=30K)");   ax6[1,   2].set_ylabel("Ra");   ax6[1,
2].set_yscale("log")
```

```
    for r in range(0, 2):
        for c in range(0, 3):
            ax6[r, c].set_xlabel("T [K]"); ax6[r, c].grid(alpha=0.3)
    fig6.suptitle("Figure 6 - Liquid Metal Properties and Rayleigh Number")
    fig6.tight_layout()
    fig6.savefig("fig06_liquid_props.png", dpi=130)


    # ---- Figure 7: cooling temperature, all groups in one plot ----
    fig7, ax7 = plt.subplots(figsize=(10, 6))
    ax7.plot(cool_time, bath_history, color="black", linestyle="--", linewidth=2.0, label="T_bath")
    for (i, j) in corner_nodes:
        ax7.plot(cool_time, cool_T_history[:, i, j], color="red", linewidth=1.4)
    for (i, j) in side_nodes:
        ax7.plot(cool_time, cool_T_history[:, i, j], color="blue", linewidth=1.2)
    for (i, j) in interior_nodes:
        ax7.plot(cool_time, cool_T_history[:, i, j], color="green", linewidth=1.4)
    ax7.axhline(600.0, color="red", linestyle=":", linewidth=1.5)
    ax7.set_xlabel("time [s]"); ax7.set_ylabel("T [K]")
    ax7.set_title("Figure 7 - Cooling: All Node Groups (overlap)")
    ax7.grid(alpha=0.3)
    ax7.legend()
    fig7.tight_layout()
    fig7.savefig("fig07_cool_all.png", dpi=130)


    # ---- Figure 8: convection coefficient h(t) for edge and side nodes ----
    n_h = len(htop_history)
    fig8, ax8 = plt.subplots(1, 2, figsize=(14, 5))
    for (i, j) in corner_nodes:
        ax8[0].plot(cool_time[0:n_h], htop_history[:, i, j], linewidth=1.8,
                    label="Node(" + str(i) + "," + str(j) + ")")
    ax8[0].set_xlabel("time [s]"); ax8[0].set_ylabel("h_top [W/m2 K]")
    ax8[0].set_title("(a) Edge (Corner) Nodes"); ax8[0].grid(alpha=0.3); ax8[0].legend(fontsize=9)
    for (i, j) in side_nodes:
        ax8[1].plot(cool_time[0:n_h], htop_history[:, i, j], linewidth=1.8,
```

```
                    label="Node(" + str(i) + "," + str(j) + ")")
    ax8[1].set_xlabel("time [s]"); ax8[1].set_ylabel("h_top [W/m2 K]")
    ax8[1].set_title("(b)     Side     Nodes");     ax8[1].grid(alpha=0.3);
ax8[1].legend(fontsize=9, ncol=2)
    fig8.suptitle("Figure 8 - Convection Coefficient h(t) During Cooling")
    fig8.tight_layout()
    fig8.savefig("fig08_h_combined.png", dpi=130)

    print("\nAll figures saved.")
    plt.show()
```

```
    # Node diagram in methodology
    import matplotlib.pyplot as plt
    import matplotlib.patches as mpatches
    import matplotlib.lines as mlines

    dx = 3.0;  dy = 4.0
    nx = 3;    ny = 5
    Lx = 6.0;  Ly = 16.0

    x_nodes = [0.0, 3.0, 6.0]
    y_nodes = [0.0, 4.0, 8.0, 12.0, 16.0]

    x_cv_lines = [0.0, 1.5, 4.5, 6.0]
    y_cv_lines = [0.0, 2.0, 6.0, 10.0, 14.0, 16.0]

    CV_FILL = "#FDEBD0"
    CV_EDGE = "#E67E22"
    C_RED   = "#C0392B"
    C_BLUE  = "#1A5276"
    C_GREEN = "#1E8449"
    C_TEXT  = "#1C2833"

    def node_color(i, j):
        corner = (i in (0, nx-1)) and (j in (0, ny-1))
        side   = (i in (0, nx-1)) or  (j in (0, ny-1))
        if corner: return C_RED,   "Edge\n(Corner)"
        if side:   return C_BLUE,  "Side"
        return             C_GREEN, "Interior"
```

```
    q_vals = {(0, 0): 32, (2, 0): 35, (0, 4): 28, (2, 4): 37}

    #figure
    fig, ax = plt.subplots(figsize=(7.5, 12))
    ax.set_xlim(-2.8, 9.0)
    ax.set_ylim(-2.5, 20.2)  # Alt sınır -3.8'den -2.5'e çekilerek boşluk
azaltıldı
    ax.set_aspect("equal")
    ax.axis("off")
    fig.patch.set_facecolor("white")

    # -title
    ax.text(3.0, 19.6,
            "2D Nodal Network \u2013 Control Volumes  (Grid: 3\u00d75)",
            ha="center",    va="top",    fontsize=11,    fontweight="bold",
color=C_TEXT)
    ax.text(3.0, 19.05,
            "Lx = 6 cm,  Ly = 16 cm,  Lz = 1 cm",
            ha="center", va="top", fontsize=9.5, color=C_TEXT)

    # cv
    plate = mpatches.Rectangle(
        (0, 0), Lx, Ly,
        facecolor=CV_FILL, edgecolor=CV_EDGE, linewidth=1.8, zorder=1)
    ax.add_patch(plate)

    for xb in x_cv_lines:
        ax.plot([xb, xb], [0, Ly], color=CV_EDGE, lw=1.0, zorder=2)
    for yb in y_cv_lines:
        ax.plot([0, Lx], [yb, yb], color=CV_EDGE, lw=1.0, zorder=2)

    # nodes
    for i, xi in enumerate(x_nodes):
        for j, yj in enumerate(y_nodes):
            clr, lbl = node_color(i, j)
            ax.plot(xi, yj, "o", color=clr, markersize=10, zorder=5,
                    markeredgecolor="white", markeredgewidth=0.6)

            if i == 0:
                tx = xi - 0.45;  ha = "right"
            elif i == nx - 1:
```

```
                tx = xi + 0.45;  ha = "left"
            else:
                tx = xi - 0.45;  ha = "right"

            ax.text(tx, yj + 0.30, f"({i},{j})\n{lbl}",
                    fontsize=7,    ha=ha,    va="bottom",    color=C_TEXT,
linespacing=1.3)

    # q'''

    q_pos = {
        (0, 4): (-1.0,  0.35, "center"),   # sol-üst köşe
        (2, 4): ( 1.0,  0.35, "center"),   # sağ-üst
        (0, 0): (-1.0, -1.15, "center"),   # sol-alt
        (2, 0): ( 1.0, -1.15, "center"),   # sağ-alt
    }
    for (i, j), q in q_vals.items():
        xi = x_nodes[i];  yj = y_nodes[j]
        ox, oy, ha = q_pos[(i, j)]
        ax.text(xi + ox, yj + oy,
                f"q'''={q} kW/m\u00b3",
                fontsize=8.5, ha=ha, color=C_RED, fontweight="bold",
                bbox=dict(boxstyle="round,pad=0.28", facecolor="#FDEDEC",
                          edgecolor=C_RED, linewidth=1.0))

    # double-headed
    def darrow(ax, x1, y1, x2, y2, label, lcolor="black",
                lx_off=0, ly_off=0.28, fs=8.5, rot=0):
        ax.annotate("", xy=(x2, y2), xytext=(x1, y1),
                    arrowprops=dict(arrowstyle="<->", color=lcolor,
                                    lw=1.1, mutation_scale=12))
        mx = (x1 + x2) / 2 + lx_off
        my = (y1 + y2) / 2 + ly_off
        ax.text(mx, my, label, ha="center", va="bottom",
                fontsize=fs, color=lcolor, rotation=rot)

    darrow(ax, 0, 17.5, 3, 17.5, "\u0394x = 3 cm")
    darrow(ax, 3, 17.5, 6, 17.5, "\u0394x = 3 cm")

    darrow(ax, -1.3, 4, -1.3, 8,  "\u0394y = 4 cm",
```

```
            lx_off=-0.40, ly_off=0, fs=8, rot=90)
    darrow(ax, -1.3, 0, -1.3, 4,  "\u0394y = 4 cm",
            lx_off=-0.40, ly_off=0, fs=8, rot=90)

    darrow(ax, 0.62, 0, 0.62, 2,
           "\u0394y/2", lcolor=CV_EDGE,
           lx_off=-0.42, ly_off=0, fs=8, rot=90)

    darrow(ax, 0, -1.1, 1.5, -1.1,
           "\u0394x/2", lcolor=CV_EDGE, ly_off=0.22, fs=8)

    darrow(ax, 0, -1.8, 6, -1.8, "Lx", ly_off=0.22, fs=9)

    darrow(ax, 7.2, 0, 7.2, 16, "Ly",
           lx_off=0.42, ly_off=0, fs=9, rot=90)


    info = ("CV Volumes:\n"
            "Corner  : (\u0394x/2)\u00b7(\u0394y/2)\u00b7Lz\n"
            "Side(s) : (\u0394x/2)\u00b7\u0394y\u00b7Lz\n"
            "Side(y) : \u0394x\u00b7(\u0394y/2)\u00b7Lz\n"
            "Interior: \u0394x\u00b7\u0394y\u00b7Lz")
    ax.text(5.9, 9.5, info,
            fontsize=7, va="center", ha="right", linespacing=1.55,
            bbox=dict(boxstyle="round,pad=0.4", facecolor="#EBF5FB",
                      edgecolor="steelblue", linewidth=0.9))

    leg_handles = [
        mlines.Line2D([], [], marker="o", linestyle="None",
                      markerfacecolor=C_RED,   markeredgecolor="white",
                      markersize=10,  label="Edge/Corner    (4  nodes,  q'''
source)"),
        mlines.Line2D([], [], marker="o", linestyle="None",
                      markerfacecolor=C_BLUE,  markeredgecolor="white",
                      markersize=10, label="Side            (8 nodes)"),
        mlines.Line2D([], [], marker="o", linestyle="None",
                      markerfacecolor=C_GREEN, markeredgecolor="white",
                      markersize=10, label="Interior        (3 nodes)"),
        mpatches.Patch(facecolor=CV_FILL, edgecolor=CV_EDGE,
                       linewidth=1.2, label="Control Volume (CV)"),
```

```
    ]
    ax.legend(handles=leg_handles, loc="lower center",
              bbox_to_anchor=(0.42, -0.11), fontsize=8.2,
              framealpha=0.95, ncol=1)

    ax.text(3.0, -2.3, "X Dimension (cm)",
            ha="center", va="center", fontsize=10, color=C_TEXT)
    ax.text(-2.6, 8.0, "Y Dimension (cm)",
            ha="center",     va="center",     fontsize=10,     color=C_TEXT,
rotation=90)

    plt.tight_layout()
    plt.savefig("node_diagram_final.png", dpi=160,
                bbox_inches="tight", facecolor="white")
    plt.show()
```

```
    # Temperature distrubition

    import numpy as np
    import matplotlib.pyplot as plt



    T_map = np.array([
        [1033.2, 1022.1, 1034.3],  # y = 0 (Bottom nodes with clamps)
        [1009.4, 1008.2, 1009.1],  # y = 1
        [1002.3, 1001.8, 1002.1],  # y = 2 (Center interior nodes)
        [1008.5, 1007.1, 1008.3],  # y = 3
        [1029.0, 1021.0, 1034.3]   # y = 4 (Top nodes with clamps)
    ])

    fig, ax = plt.subplots(figsize=(6, 8))

    im  =  ax.imshow(T_map,  cmap='jet',  origin='lower',  aspect='auto',
alpha=0.85)

    cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label('Temperature     [K]',     rotation=270,     labelpad=20,
fontsize=12, fontweight='bold')

    # Configures
    ax.set_xticks(np.arange(3))
```

```
    ax.set_yticks(np.arange(5))
    ax.set_xticklabels(['x = 0', 'x = 1', 'x = 2'], fontsize=11)
    ax.set_yticklabels(['y = 0', 'y = 1', 'y = 2', 'y = 3', 'y = 4'],
fontsize=11)
    ax.set_xlabel('X Nodes (Horizontal)', fontsize=12, fontweight='bold',
labelpad=10)
    ax.set_ylabel('Y Nodes (Vertical)', fontsize=12, fontweight='bold',
labelpad=10)
    ax.set_title('2D Temperature Distribution Map\nEnd of Heating Phase',
                 fontsize=13, fontweight='bold', pad=15)

    for i in range(5):
        for j in range(3):
            text_color = "white" if (T_map[i, j] > 1025 or T_map[i, j] <
1005) else "black"

            ax.text(j, i, f"{T_map[i, j]:.1f} K",
                    ha="center", va="center", color=text_color,
                    fontweight="bold", fontsize=12)

            ax.text(j, i-0.2, f"Node({j},{i})",
                    ha="center", va="center", color=text_color,
                    fontsize=9, alpha=0.9)

    ax.set_xticks(np.arange(-.5, 3, 1), minor=True)
    ax.set_yticks(np.arange(-.5, 5, 1), minor=True)
    ax.grid(which="minor", color="black", linestyle='-', linewidth=2)
    ax.tick_params(which="minor", bottom=False, left=False)

    plt.tight_layout()
    plt.savefig("temperature_heatmap.png", dpi=160, bbox_inches='tight')
    plt.show()
```